

OpenShift Virtualization cookbook

Version 1, 2026-06-12

目次

目次

OpenShift のインストールとアップグレードの概要

インストールYAMLとCLIのインストールとアップグレードKubernetes/OpenShift
インストールとアップグレードの概要

(日付: {build-date})

目次

インストールとアップグレードの概要

- 対象: OpenShift 4.18+ の OpenShift のインストールとアップグレード
- 対象: `oc`, `kubectl`, `virtctl` のインストール: `kustomize`, `helm`, `ansible`
- 対象: YAML のインストールとCLIのインストールとアップグレードKubernetes/OpenShift のインストールとアップグレード

インストールとアップグレードの概要

目次

インストール 1 のインストールとアップグレード

- `oc` - インストールとアップグレード
- `kubectl` - インストールとアップグレード
- `virtctl` - インストールとアップグレード IP のインストール
- `kustomize` - VM のインストールDNSのインストールとアップグレード
- `VM` のインストールとアップグレード - CPU のインストールNUMAのインストールとアップグレードhugepagesのインストールとアップグレード VM
- `VM` のインストールとアップグレード - インストールとアップグレードのインストールとアップグレード
- `API` のインストール - インストールとアップグレードのインストールとアップグレード API のインストールとアップグレード

目次

- OpenShift のインストール
 - `oc`
 - `kubectl`
 - `kustomize` (pdf)
 - `VM` のインストールとアップグレード

- □ □ □ □ □ □ □ □ □

11

[illegible]

--	--	--	--	--

OpenShift 2020 Alexandre Menezes, Andrea Perotti, Daphne Hoover, Josh Manning, Melvin Hillsman, Roshan Warriar, Yash Devan Oza

[illegible]

□ □ □ □ □

MIT 00000 0000000000000000

[illegible]

Cloud-init 設定

Cloud-init は VirtualMachine 上で cloud-init を実行する

インストール

インストール方法

- [Cloud-init](#) - Cloud-init のインストール
- [Cloud-init](#) - VM のインストール
- **Cloud-init** のインストール - cloud-init のインストール

設定

設定

Cloud-init は OpenShift のインストール時に実行される

インストール

OpenShift のインストール

- OpenShift 4.18 のインストール
- OpenShift のインストール

インストール

インストール

- インストール: OpenShift のインストール
- インストール: OpenShift のインストール
- インストール: OpenShift のインストール

OpenShift CLI のインストール

OpenShift CLI (oc)

OpenShift のインストール

OpenShift CLI のインストール

インストール

```
oc version
```

Kubernetes CLI (kubectl)

`oc` の代わりに `kubectl` を使ってみよう

Kubernetes のインストール

インストール

```
kubectl version
```

virtctl

KubeVirt/OpenShift のインストール

OpenShift のインストール

インストール

```
virtctl version
```

インストール

インストール

- **kustomize**: Kubernetes のインストール
- **helm**: Kubernetes のインストール
- **ansible**: Kubernetes のインストール

インストール

インストール

インストール

- 1 つの StorageClass をインストール
- VM をインストール
- Kubernetes のインストール

StorageClass のインストール

```
oc get storageclass
```

インストール

Pod のインストール

[HyperConverged] | *install-8-click-create.png*

4:

HyperConverged

[HyperConverged] | *install-9-wait-for-conditions.png*

`Operators` ` ` ` `

[OpenShift] | *install-10-installed-operator.png*

Web CLI

1: **Namespace** **OperatorGroup** **Subscription**

1:

```
oc apply -f - <<EOF
apiVersion: v1
kind: Namespace
metadata:
  name: openshift-cnv
---
apiVersion: operators.coreos.com/v1
kind: OperatorGroup
metadata:
  name: kubevirt-hyperconverged-group
  namespace: openshift-cnv
spec:
  targetNamespaces:
  - openshift-cnv
---
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: hco-operatorhub
  namespace: openshift-cnv
spec:
  source: redhat-operators
  sourceNamespace: openshift-marketplace
  name: kubevirt-hyperconverged
  channel: stable
  installPlanApproval: Automatic
EOF
```

ステップ2: 確認コマンドを実行

確認コマンドを実行:

```
# CSVがインストールされたことを確認
oc get csv -n openshift-cnv -w
```

確認コマンドを実行:

NAME	DISPLAY	VERSION
REPLACES	PHASE	
kubvirt-hyperconverged-operator.v4.20.3	OpenShift Virtualization	4.20.3
kubvirt-hyperconverged-operator.v4.20.1	Succeeded	

`Succeeded`を確認し、Ctrl+Cで確認を中止

ステップ3: **HyperConverged**をインストール

HyperConvergedをインストール:

```
oc apply -f - <<EOF
apiVersion: hco.kubvirt.io/v1beta1
kind: HyperConverged
metadata:
  name: kubvirt-hyperconverged
  namespace: openshift-cnv
spec:
EOF
```

注: HyperConvergedのインストールは、OpenShift Virtualization 4.20.3以降のバージョンでサポートされています。

ステップ4: 確認コマンドを実行

HyperConvergedのインストールを確認:

```
oc wait hyperconverged kubvirt-hyperconverged -n openshift-cnv
--for=condition=Available --timeout=15m
```

ステップ5-10: 確認コマンドを実行

ステップ5: 確認コマンドを実行

確認コマンドを実行:

```
oc get hyperconverged kubvirt-hyperconverged -n openshift-cnv -o jsonpath='{range
```

```
.status.conditions[*]{}.type: {}.status{"\n"}{end}'
```

□□□□□□□□□□□□□□□□:

```
ReconcileComplete: True
Available: True
Progressing: False
Degraded: False
Upgradeable: True
```

OpenShift

OpenShift
OpenShift

HyperConverged

0000**1**: 000000000000

VM

```
# 仮想マシンVMを削除
oc get vm -A

# 仮想マシンVMを削除 (namespace)
oc delete vm --all -n <namespace>
```

00: 0000000000000000VM000000000000

2: HyperConverged

OpenShift:

```
oc delete hyperconverged kubevirt-hyperconverged -n openshift-cnv
```

□□□□□□□□□□□□:

```
# 5-10초 (openshift-cnvruntime)
oc get pods -n openshift-cnvruntime -w
```

[illegible]

ステップ3: オペレーターハブを削除する

```
# オペレーターハブを削除
oc delete subscription hco-operatorhub -n openshift-cnv

# ClusterServiceVersionを削除
oc delete csv -n openshift-cnv -l operators.coreos.com/kubevirt-
hyperconverged.openshift-cnv
```

ステップ4: 名前空間を削除する

名前空間を削除する:

```
oc delete namespace openshift-cnv
```

ステップ5: 確認する

確認する:

```
# "No resources found"を確認
oc get pods -n openshift-cnv
oc get csv -n openshift-cnv
```

完了

OpenShiftインストール完了

インストール

- [virtctl](#) - VMを管理するためのツール
- [libvirt](#) - VMを管理するためのライブラリ
- [VM IP](#) - cloud-init/DHCPで自動的に割り当てられるIPアドレス

完了

Single Node OpenShiftインストール完了

- [OpenShift](#)
- [OpenShift](#)
- [KubeVirt](#)

インストール完了

VOLUMEBINDINGMODE	ALLOWVOLUMEEXPANSION	AGE	
ceph-rbd-sc (default)	rbd.csi.ceph.com		Delete
Immediate	true	2d	
hostpath-provisioner	kubevirt.io/hostpath-provisioner		Delete
WaitForFirstConsumer	false	1d	
local-storage	kubernetes.io/no-provisioner		Delete
WaitForFirstConsumer	false	1d	

ローカルストレージ (default) のデフォルト設定

ローカルストレージのデフォルト設定

ローカルストレージのデフォルト設定を PVC に適用

```
oc apply -f - <<EOF
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: test-default-storage
  namespace: default
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
EOF
```

ローカルストレージのデフォルト設定を PVC に適用

```
oc get pvc test-default-storage -n default
```

ローカルストレージ

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS	AGE
test-default-storage	Pending				lvms-vg1	0s

PVC は Pending の状態で (lvms-vg1) のストレージを待っています

ローカルストレージのデフォルト設定

PVC は Pending の状態で WaitForFirstConsumer のストレージを待っています

StorageClass	Pod	StorageClass
WaitForFirstConsumer	Pod → PVC → StorageClass → PV StorageClass	Pod StorageClass LVM
Immediate	PVC → StorageClass → PV → StorageClass	StorageClass → NFS → Ceph → StorageClass

StorageClass

```
oc get sc <storage-class-name> -o jsonpath='{.volumeBindingMode}'
```

StorageClass 6 → VM

StorageClass

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: test-vm-default-storage
  namespace: default
spec:
  runStrategy: Always
  dataVolumeTemplates:
  - metadata:
      name: test-vm-disk
    spec:
      pvc:
        accessModes:
        - ReadWriteOnce
        resources:
          requests:
            storage: 10Gi
        sourceRef:
          kind: DataSource
          name: fedora
          namespace: openshift-virtualization-os-images
      template:
        metadata:
          labels:
            kubevirt.io/vm: test-vm-default-storage
        spec:
          domain:
            devices:
              disks:
                - name: datavolumedisk
                  disk:
```



```

    bus: virtio
  resources:
    requests:
      memory: 1Gi
      cpu: 1
  networks:
  - name: default
    pod: {}
  volumes:
  - name: datavolumedisk
    dataVolume:
      name: test-vm-disk
EOF
```

[illegible]

☐ ☐ ☐ ☐ ☐ ☐

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

A horizontal bar containing numerous small, identical square icons arranged side-by-side.

```
oc get sc -o custom-  
columns=NAME:.metadata.name,DEFAULT:.metadata.annotations.storageclass\.kubernetes\.io  
/is-default-class
```

[illegible][illegible]

```
oc get sc <storage-class-name>
```

[illegible][illegible]

```
oc auth can-i create storageclass --all-namespaces
```

[Home](#) [About Us](#) [Contact Us](#) [Privacy Policy](#) [Terms & Conditions](#)

[illegible]

```
oc get sc -o wide
```

































```
oc describe sc <storage-class-name>
```

다음은 PVC 정보를 출력하는 명령어입니다.

```
oc get pvc --all-namespaces -o custom-  
columns=NAMESPACE:.metadata.namespace,NAME:.metadata.name,STORAGECLASS:.spec.storageCl  
assName
```

다음은 PVC 정보를 출력하는 명령어입니다.

1. PVC 정보를 출력하는 명령어입니다.
2. PVC 정보를 출력하는 명령어입니다.
3. PVC 정보를 출력하는 명령어입니다.
4. PVC 정보를 출력하는 명령어입니다.

다음은 PVC 정보를 출력하는 명령어입니다.

다음은 PVC 정보를 출력하는 명령어입니다.

- PVC 정보를 출력하는 명령어입니다.
- storageclass.kubernetes.io/is-default-class 정보를 출력하는 명령어입니다.
- PVC 정보를 출력하는 명령어입니다.
- OpenShift Virtualization 정보를 출력하는 명령어입니다.

다음은 PVC 정보를 출력하는 명령어입니다.

- [OpenShift - PVC 정보를 출력하는 명령어](#)
- [OpenShift Virtualization - PVC 정보를 출력하는 명령어](#)

Web 콘솔에서 VM 정보

다음은 Web 콘솔에서 VM 정보를 출력하는 명령어입니다.

다음은 Web 콘솔에서 VM 정보를 출력하는 명령어입니다.

다음은 Web 콘솔에서 VM 정보를 출력하는 명령어입니다.

- OpenShift 4.18에서 OpenShift Virtualization 정보를 출력하는 명령어
- OpenShift Web 콘솔에서 VM 정보를 출력하는 명령어

- VM `StorageClass`
- `DataSources`

OpenShift Virtualization

```
oc get datasources -n openshift-virtualization-os-images
```

Step 1: Access the OpenShift console

1. OpenShift Web Console
2. OpenShift Virtualization
3. **VirtualMachines**

[OpenShift VirtualMachines] | *virtualization-menu.png*

VirtualMachines

Step 2: Select a template

OpenShift Virtualization

1. VirtualMachines **Create** → **From template**
2. Select a template
 - **Red Hat Enterprise Linux** - RHEL
 - **Fedora** - Fedora
 - **CentOS Stream** - CentOS Stream
 - **Microsoft Windows** - Windows Server

[OpenShift Virtualization] | *vm-templates-catalog.png*



Source

available `StorageClass`

OpenShift Virtualization

- CPU
- OS
- `DataSource`
- `cloud-init` `sysprep`

Step 3: Select a VM

OpenShift Fedora VM

□ □ □ □ □ □ □ □ □

1. **Fedora**□□□□□□□□□□□□□□□□
2. □□□□□□□□□□□□□□□□□□□□
3. **Customize VirtualMachine**□□□□□□□□

[Fedora] | *fedora-template-details.png*

VM□□□□□□

Customize VirtualMachine Overview VM

1. **Name:** VMXXXXXXXXXXXXfedora-test-vm
2. **Project:** VMXXXXXXXXXXXXnamespaceXXXXXX
3. **Boot source:** Fedora DataSourceXXXXXXXXXXXX
4. **CPU:** CPUXXXXXXXXXXXX1XX
5. **Memory:** RAMXXXXXXXXXXXX

[VM Overview] | *vm-customize-overview.png*

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

Disks

1. 環境構築
2. 開発環境構築
3. StorageClassの作成

[VM - Disks] | *vm-customize-disks.png*

□ □ □ □ □ □

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

1. Create VirtualMachine
2. VMVM

[VirtualMachine□□□□□] | *vm-create-button.png*

00004VM00000000

Create VM

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

VM

1. 00000000 - 00000000000000000000

2. **Starting** - VM

3. **Running** - VM

[VM] | *vm-provisioning-status.png*

Events

```
# CLI
oc get events -n <your-namespace> --field-selector involvedObject.name=<vm-name> -w
```

VM

Running

[VM] | *vm-running-status.png*

5 VM

Web VM

1. VM **Console**

2. VNC

[VM/] | *vm-console-tab.png*

- **Send key** - Ctrl+Alt+Del
- **Disconnect** -
- **Guest credentials** -



Linux
VM RDP Web

VM SSH Windows

Fedora cloud-init

- cloud-init
-
- cloud-init

第6章 VM

Web VM

操作

VM VirtualMachines

[VM] | *vm-actions-menu.png*

VM

1. **Actions** → **Stop**
- 2.
3. VM

```
# CLI  
virtctl stop <vm-name>
```

VM

1. **Actions** → **Restart**
2. VM

```
# CLI  
virtctl restart <vm-name>
```

操作

VM

1. **Actions** → **Pause**
2. VM
3. **Actions** → **Unpause**



VM

VM

1. **Actions** → **Delete**
2. PVC
- 3.

[VM] | *vm-delete-confirmation.png*



☑️ **Delete**

disk VM **DataVolume** VM



VM

7 VM

VM

- VM
- IP
- CPU
-

[VM] | *vm-overview-tab.png*

- CPU
-
-
- I/O

[VM] | *vm-metrics-tab.png*



QEMU VM

YAML

VM

1. **YAML**
2. **VirtualMachine**
3. **Save**

[VM YAML] | *vm-yaml-tab.png*



YAML **Download**

○○○○○○○○○○○○**VM**○○○○○○

○○○○○○○○○○○○**VM**○○○○○○○○○○○○○○○○○○○○○○○○

1. VirtualMachines○○○○○○**Create** → **From template**○○○○○○○○
2. ○○○○○○○○○○○
3. **Customize VirtualMachine**○○○○○**Quick create VirtualMachine**○○○○○○○○
4. ○○○○○○○○○**VM**○○○○○○○○○○

[○○○○○○○○VirtualMachine○○○○○] | *vm-quick-create.png*

○○○○○○○○○○○○○○○○○○

- ○○○○○○○○○
- ○○○○○○○○○○○○○○○○○○○○○
- ○○○○○○○○○

□□

○○○○○○○○○○

- Web○○○○○○○○○○○○○○○○○○○○
- VM○○○○○○○○○○○○○○
- ○○○○○○○○○○○**VM**○○○○
- **VM**○○○○○○○○○○○○○○○○
- Web○○○○○**VM**○○○○○○○○○○○○○○○○
- ○○○**VM**○○○○○○○○○○○○○○○○○○
- **VM**○○○○○○○○○○○○

□□□□□

- **virtctl CLI**□□□ - ○○○○○○○**VM**□□
- **IP**□□□ - **VM**□□□**IP**□□
- **Linux VM**□□**SSH**□□□□□□ - **SSH**□□□□**Linux VM**□□□□

□□

- **OpenShift Container Platform**○○○○○○○○□□ - ○○○○○○○□□
- **OpenShift Container Platform**○○○○○○○○□□ - ○○○○○○○○○○○□□

VM ○○○○○○○○○○○

OpenShift

VirtualMachine

[illegible]

□ □ □ □

- ## VirtualMachine $\square$ VirtualMachineInstance

VirtualMachine (VM)

000CPU

[illegible]

□□□:

VirtualMachineInstance (VMI)

□□□□□□

```
# VirtualMachineInstance
oc get vmi -n my-vms
```

결과:

NAME	AGE	PHASE	IP	NODENAME	READY
fedora-app-vm	78s	Running	10.128.0.184	ocplab01	True



VirtualMachine
VirtualMachineInstance
VMI
VirtualMachine

VM

목록

번호	이름	설명	VMI
1	기본	기본 VM	기본
2	기본	기본 (기본) VM	기본
3	기본	Kubernetes	기본
4	기본	기본	기본
5	기본	기본	기본
6	기본	VM	기본
7	기본	기본	기본
8	기본	VMI	기본 (기본)

VM

목록

목록:

Web

1. 기본 → 기본
2. 기본 VM
3. VM 기본 → 기본
4. 기본3 기본 (기본) 기본

CLI:

```
virtctl start fedora-app-vm -n my-vms
```

確認:

```
# VM 確認
oc get vm fedora-app-vm -n my-vms

# VMI 確認
oc get vmi fedora-app-vm -n my-vms
```

例:

NAME	AGE	STATUS	READY
fedora-app-vm	14m	Running	True

NAME	AGE	PHASE	IP	NODENAME	READY
fedora-app-vm	78s	Running	10.128.0.184	ocplab01	True

確認

仮想マシンが再起動されたことを確認するVMIを確認

Web 確認:

1. VM 確認
2. 確認 → 確認

CLI (確認):

```
virtctl stop fedora-app-vm -n my-vms
```

CLI (確認):

VM 確認:

```
virtctl stop fedora-app-vm -n my-vms --force
```



仮想マシンを強制停止する

확인합니다:

```
# VM 확인
oc get vm fedora-app-vm -n my-vms

# VMI 확인
oc get vmi fedora-app-vm -n my-vms
```

결과:

NAME	AGE	STATUS	READY
fedora-app-vm	14m	Stopped	False

Error from server (NotFound): virtualmachineinstances.kubevirt.io "fedora-app-vm" not found

확인합니다

확인합니다 VM 확인합니다

- CPU 확인합니다
- 확인합니다
- 확인합니다

결과: 확인합니다 CPU 확인합니다

Web 확인:

1. VM 확인
2. 확인 → 확인
3. 확인 → 확인

[확인] | [vm-lifecycle-states/vm-running-pause-action.png](#)

[확인] | [vm-lifecycle-states/vm-paused-unpause-action.png](#)

CLI:

```
# VM 확인
virtctl pause vm fedora-app-vm -n my-vms

# VM 확인
virtctl unpause vm fedora-app-vm -n my-vms
```

확인합니다:

```
oc get vm fedora-app-vm -n my-vms
```

결과:

NAME	AGE	STATUS	READY
fedora-app-vm	14m	Paused	False

문제점

다음과 같은 문제가 발생할 수 있습니다:

- VM OS 이미지 손상
- VM OS 이미지 크기 부족
- VM 이미지 삭제 실패

Web 콘솔:

1. VM 콘솔에 접속
2. 콘솔 → VM 콘솔

[콘솔 화면] | [vm-lifecycle-states/vm-running-restart-action.png](#)

CLI:

```
virtctl restart fedora-app-vm -n my-vms
```



다음과 같은 문제가 발생할 수 있습니다 OS 이미지 크기 부족

문제점

다음과 같은 문제가 발생할 수 있습니다 (PVCs/DataVolumes) 이미지 크기 부족

Web 콘솔:

1. VM 콘솔에 접속
2. 콘솔 → VM 콘솔
3. 콘솔 화면에서 이미지 크기 확인
4. VM 콘솔

[콘솔 화면] | [vm-lifecycle-states/vm-stopped-delete-action.png](#)

[콘솔 화면] | [vm-lifecycle-states/vm-delete-confirmation-dialog.png](#)

CLI (콘솔 화면에서):

```
oc delete vm fedora-app-vm -n my-vms
```

VM DataVolume Web
:

```
# VM  
oc get vm fedora-app-vm -n my-vms  
  
# ( )  
oc get pvc -n my-vms
```

:

```
Error from server (NotFound): virtualmachines.kubevirt.io "fedora-app-vm" not found  
my-vms
```



VM

VM

VM **VMI**

VM :

```
oc describe vm fedora-app-vm -n my-vms
```

():

```
Name:          fedora-app-vm  
Namespace:     my-vms  
Labels:        app=fedora-app-vm  
...  
Status:  
  Conditions:  
    Last Transition Time:  2026-03-05T17:10:11Z  
    Status:               True  
    Type:                 Ready  
    Message:              All of the VMI's DVs are bound and ready  
    Reason:               AllDVsReady  
    Status:               True  
    Type:                 DataVolumesReady  
    Status:               True  
    Type:                 AgentConnected
```

```
Printable Status:    Running
Ready:               true
Run Strategy:        Manual
```

□□□□□□□□:

- **Status.Conditions** - □□□□□□□□ (□□□□) □□□□□
- **Status.PrintableStatus** - □□□□□□□□□□□□
- **Status.Ready** - VM □□□□□□□□□

□□□ VMI □□:

```
oc describe vmi fedora-app-vm -n my-vms
```

□□□ (□□□□):

```
Name:                fedora-app-vm
Namespace:           my-vms
...
Status:
  Guest OS Info:
    Id:               fedora
    Kernel Release:   6.17.1-300.fc43.x86_64
    Name:             Fedora Linux
    Pretty Name:      Fedora Linux 43 (Cloud Edition)
    Version Id:       43
  Interfaces:
    Interface Name:   enp1s0
    Ip Address:       10.128.0.184
    Mac:              02:7e:e2:0a:7c:df
    Name:             default
  Node Name:          ocplab01
  Phase:              Running
  Phase Transition Timestamps:
    Phase: Pending
    Phase: Scheduling
    Phase: Scheduled
    Phase: Running
```

□□□□□□□□:

- **Status.Phase** - □□□□□□□□□□□□□□
- **Status.Interfaces** - IP □□□□□□□□□□□□□□
- **Status.NodeName** - VM □□□□□□□□□□
- **Status.GuestOSInfo** - □□□ OS □□□ (□□□□□□□□□□□□□□□□□□□□□□)

[illegible]

□□□:

□□□□□□:

VM □□□□

31


```
oc get events -n my-vms --field-selector involvedObject.name=fedora-app-vm
```

000:

LAST SEEN	TYPE	REASON	OBJECT
14m	Normal	SuccessfulDataVolumeCreate	virtualmachine/fedora-app-vm
Created DataVolume fedora-app-vm-rootdisk			
90s	Normal	SuccessfulCreate	virtualmachine/fedora-app-vm
Started the virtual machine by creating the new virtual machine instance fedora-app-vm			
90s	Normal	SuccessfulCreate	virtualmachineinstance/fedora-app-vm
Created virtual machine pod virt-launcher-fedora-app-vm-fktmb			
84s	Normal	Created	virtualmachineinstance/fedora-app-vm
VirtualMachineInstance defined.			
84s	Normal	Started	virtualmachineinstance/fedora-app-vm
VirtualMachineInstance started.			

[illegible]

```
oc get events -n my-vms -w
```

[illegible]

成功	成功
SuccessfulCreate	VMI 成功创建虚拟机
Started	VM 成功启动虚拟机
Stopped	VM 成功停止虚拟机
Killing	成功杀死虚拟机
FailedScheduling	虚拟机调度失败
FailedCreate	VMI 创建虚拟机失败 (创建失败)

□ □ □ □

[illegible]

□ □ □ □ □ □ □ □ □

項目	内容	備考
Always	VM 仮想マシン 仮想マシン	仮想マシン

状態	操作	説明
RerunOnFailure	VM 失敗した VM を自動的に再起動する	失敗した VM を自動的に再起動する
Manual	VM から Web コンソールで virtctl 操作を行う	VM から Web コンソールで VM を操作する
Halted	VM を手動で停止する	失敗した VM を手動で停止する

例

YAML 例:

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: my-vm
spec:
  runStrategy: Always # 状態 RerunOnFailure/Manual/Halted
  template:
    # ... VM の定義
```

Web コンソール:

1. VM を作成する
2. **YAML** を定義する
3. **spec.runStrategy** を設定する
4. 操作を実行する

[YAML 例の **runStrategy** の設定] | [vm-lifecycle-states/vm-yaml-run-strategy.png](#)



runStrategy を設定して **running** 状態の VM を **runStrategy** で操作する

例

Always 状態 **RerunOnFailure** の場合

- **virtctl stop** で VM を停止する
- VM が **Halted** 状態になったら **Manual** で再起動する
- 再起動する (例: **shutdown -h now** で VM を再起動) する

例

例:

- VirtualMachine → VirtualMachineInstance
- VM
- VM
- VM
- VM

- [Web](#) - Web
- [virtctl](#) - VM CLI
- [window=_blank](#)

virtctl

virtctl OpenShift Virtualization VM

:

OpenShift 4.20

- OpenShift 4.18+ OpenShift Virtualization
- virtctl CLI

virtctl

virtctl CLI OpenShift Web

1. OpenShift Web →
2. OpenShift image::virtualization-overview.png
3. Overview virtctl
4. virtctl image::virtctl-download.png
5. virtctl
6. virtctl image::virtctl-versions.png
7. PATH

```
# Linux/Mac
chmod +x virtctl
sudo mv virtctl /usr/local/bin/
```

```
# 確認
virtctl version
```

Windows の PATH 環境変数

確認

確認

確認 **oc** 確認

```
# 確認
oc get vm -n myapp

# 確認
oc get vm -A
```

確認:

NAMESPACE	NAME	AGE	STATUS	READY
myapp	fedora-app-vm	13d	Running	True

確認 VM (VMI) (IP 確認):

```
oc get vmi -A
```

確認:

NAMESPACE	NAME	AGE	PHASE	IP	NODENAME	READY
myapp	fedora-app-vm	13d	Running	10.128.1.225	ocplab01	True

確認: **oc** **virtctl** VM 確認

VM 確認

確認:

```
# VM 確認
virtctl start fedora-app-vm -n myapp

# VM 確認
virtctl stop fedora-app-vm -n myapp

# VM 確認
```

```
virtctl restart fedora-app-vm -n myapp
```

VM 操作

VM 操作:

```
# 重启 VM (Ctrl+] 重启)
virtctl console fedora-app-vm -n myapp

# SSH 连接 (SSH 连接)
virtctl ssh user@fedora-app-vm -n myapp

# VNC 连接
virtctl vnc fedora-app-vm -n myapp
```

VM 信息

QEMU 虚拟机 VM 信息: VM 信息:

```
# 获取 OS 信息
virtctl guestosinfo fedora-app-vm -n myapp
```

输出:

```
{
  "guestAgentVersion": "10.1.0",
  "hostname": "fedora-app-vm",
  "os": {
    "name": "Fedora Linux",
    "kernelRelease": "6.5.6-300.fc39.x86_64",
    "version": "39",
    "prettyName": "Fedora Linux 39 (Cloud Edition)",
    "versionId": "39",
    "kernelVersion": "#1 SMP PREEMPT_DYNAMIC",
    "machine": "x86_64",
    "id": "fedora"
  }
}
```

```
# 列出文件系统
virtctl fslist fedora-app-vm -n myapp
```

输出:

```
{
```

```
"items": [
  {
    "diskName": "vda2",
    "mountPoint": "/boot/efi",
    "fileSystemType": "vfat",
    "usedBytes": 21723136,
    "totalBytes": 104607744
  },
  {
    "diskName": "vda3",
    "mountPoint": "/boot",
    "fileSystemType": "ext4",
    "usedBytes": 91316224,
    "totalBytes": 1902895104
  },
  {
    "diskName": "vda4",
    "mountPoint": "/",
    "fileSystemType": "btrfs",
    "usedBytes": 687783936,
    "totalBytes": 29525868544
  }
]
}
```

```
# 列出虚拟机中的用户
virtctl userlist fedora-app-vm -n myapp
```

虚拟机

virtctl scp 在虚拟机和 VM 之间复制文件。SSH 密钥对必须在虚拟机中：

- **SSH 公钥 (key):** VM 在 `~/.ssh/authorized_keys` 文件中。
- **私钥:** VM 在 SSH 密钥对文件中。

```
# 在本地和 SSH 虚拟机之间复制文件
virtctl scp local-file.txt fedora@fedora-app-vm:/tmp/remote.txt -n myapp

# VM 本地复制
virtctl scp fedora@fedora-app-vm:/tmp/remote.txt ./local.txt -n myapp

# 在本地和 SSH 虚拟机之间复制文件
virtctl scp -i ~/.ssh/my_key local-file.txt fedora@fedora-app-vm:/tmp/ -n myapp
```

注意: SSH 密钥对必须在虚拟机中 **virtctl console** 连接到 VM 并输入 SSH 密钥对。

VM 必须在 SSH 密钥对文件中包含 **cloud-init** 密钥对。

```
#cloud-config
user: fedora
ssh_authorized_keys:
- ssh-rsa AAAAB3NzaC... your-key-here
```

SSH 接続可能な Linux VM を作成する

VM の作成

VM を作成する:

```
# VM の作成
oc get vm fedora-app-vm -n myapp
```

確認:

NAME	AGE	STATUS	READY
fedora-app-vm	7d3h	Running	True

```
# VMI (Virtual Machine Instance) の取得
oc get vmi fedora-app-vm -n myapp
```

VMI の詳細を確認する

例

確認:

- `virtctl console` - VMI のコンソールに接続する
- `virtctl ssh` - VMI に SSH 接続する
- `virtctl vnc` - VMI の VNC 接続 (GUI での VM 操作)

VM の管理:

- `virtctl guestosinfo` - VMI の OS 情報を取得
- `virtctl fslist` - VMI のファイルシステムを確認
- `virtctl userlist` - VMI のユーザーリストを確認

まとめ

例: VMI の作成と確認

例: VMI の削除

```
oc whoami
oc get nodes
```

□□: □□□□□□□□□□□□□□□□

注意: VM の QEMU 環境で実行する場合は、**redhat** を指定してください。

SSH/SCP

注意: VM 的 SSH 密钥必须与本地环境保持一致:

```
# 虚拟机重启
virtctl console fedora-app-vm -n myapp

# VM 通过 SSH 连接虚拟机
systemctl status sshd
```

□□: □□□□□□□□□□□□□□□□

Press **[Ctrl+] VM** to start the VM:

```
oc get vmi fedora-app-vm -n myapp
```

□ □ □ □ □

- `vim` 編輯 (**-n**) 新建檔案
- 透過 `ssh` 登入遠端主機
- 安裝 **VM** 與 **QEMU** 虛擬化環境
- **VM** 安裝 (**oc**) 安裝 **VM** (**virtctl**)
- 透過 **(Ctrl+J)** 登入遠端主機
- 透過 **virtctl ssh** 登入

☐ ☐ ☐ ☐ ☐ ☐

□□□□□□□□□□□□□□:

- OpenShift Web 管理画面 virtctl CLI 管理画面
- 管理画面から VM を起動
- SSH/VNC 接続 VM 操作

- 11

- □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

11

```
Virtualization - VM - SSH  
KubeVirt - VM - VirtualMachine
```

□□□□

- `VirtualMachine`のメタデータは、`VM`のメタデータとして保存される
- `VirtualMachine`のメタデータは、`3`つのメタデータとして保存される
`userData`、`networkData`、`metadata`
- `KubeVirt`の`VM`のメタデータは、`NoCloud`のメタデータとして保存される
- `VirtualMachine`のメタデータは、`cloud-config`のメタデータとして保存される
- `VirtualMachine`のメタデータは、`2`つのメタデータとして保存される
`Secret`と`Metadata`
- `VirtualMachine`のメタデータは、`cloud-config`のメタデータとして保存される

□□□□:

□ □ □ □

- OpenShift 4.18+ OpenShift Virtualization
- `oc`virtctl` CLI `virtctl`
- YAML
- VM `Web` VM

□ □

```

#####
#####SSH#####

```

Fedora RHEL CentOS Ubuntu Debian Linux AWS
 S Azure GCP KubeVirt KubeVirt OpenShift Virtualization

0: VM

KubeVirt

VM HTTP `169.254.169.254` KubeVirt
irt *NoCloud*

NoCloud KubeVirt

1. VirtualMachine Secret
2. ISO
3. virtio VM
4. OS

Kubernetes

```
spec:
  template:
    spec:
      volumes:
        - name: cloudinitdisk
          cloudInitNoCloud: ①
            userData: |
              #cloud-config
              ssh_pwauth: true
              user:
                name: fedora
                lock_passwd: false
                plain_text_passwd: changeme
```

① `cloudInitNoCloud` NoCloud KubeVirt

3 KubeVirt `cloudInitNoCloud`

userData (cloud-config)

userData #cloud-config
config YAML VM

```
cloudInitNoCloud:
  userData: |
    #cloud-config
    ssh_pwauth: true
    user:
      name: fedora
```

```

lock_passwd: false
plain_text_passwd: s3cur3p4ss
ssh_authorized_keys:
  - ssh-ed25519 AAAA...user@workstation
packages:
  - vim
  - tmux

```

例: `cat /etc/cloud/cloud-init.yaml` `cloud-config`

networkData (network-config v2)

VM環境でNetplanの`*network-config`2*のIPとDNSを設定

例DHCP:

```

cloudInitNoCloud:
  networkData: |
    version: 2
    ethernets:
      enp1s0:
        dhcp4: true

```

例IPとDNS:

```

cloudInitNoCloud:
  networkData: |
    version: 2
    ethernets:
      enp1s0:
        addresses:
          - 192.168.1.100/24 ①
        nameservers:
          addresses:
            - 192.168.1.1
            - 8.8.8.8 ②
        routes:
          - to: default
            via: 192.168.1.1 ③
          - to: 10.0.0.0/8
            via: 192.168.1.254 ④

```

① CIDR形式のIPアドレス

② DNS resolver `<3>` `gateway4`` `<4>` `10.0.0.0/8``

例MACアドレス

```
cloudInitNoCloud:
  networkData: |
    version: 2
    ethernets:
      primary:
        match:
          macaddress: "52:54:00:12:34:56"
        addresses:
          - 192.168.1.100/24
        routes:
          - to: default
            via: 192.168.1.1
```

```
❏:
KubeVirt VM enp1s0 enp2s0 VirtualMachine
NetworkAttachmentDefinitions
```

□□□□□□□□□□VLAN□□□□□□□□□□□□IP□□□□□□□□□□

metadata

```
VM instance-id`local-  
hostname KubeVirt VirtualMachine
```

```
cloudInitNoCloud:
  metadata: |
    instance-id: my-vm-001
    local-hostname: my-vm
```

```
000: 000000000000000000000000 `hostname` 0000000000userData0000000000000000
```

□ □

[illegible]

```

networkData
userData
/`runcmd`

```

cloud-config

VM#cloud-configuserData

```
#cloud-config
ssh_pwauth: true
user:
  name: fedora
  lock_passwd: false
  plain_text_passwd: s3cur3p4ss
  ssh_authorized_keys:
    - ssh-ed25519 AAAA...key-data... user@workstation
```

`user` `lock\_passwd`:
false` `plain_text_passwd` `ssh_pwauth`:
true`SSH`

`users`

```
#cloud-config
users:
  - name: admin
    groups: wheel
    sudo: ALL=(ALL) NOPASSWD:ALL
    ssh_authorized_keys:
      - ssh-ed25519 AAAA...key-data...
  - name: developer
    shell: /bin/bash
```

```
#cloud-config
package_update: true
packages:
  - vim
  - git
  - tmux
  - python3
```

: **final**

```
#cloud-config
runcmd:
```

- systemctl enable --now cockpit.socket
- echo "VM initialized at \$(date)" >> /var/log/vm-init.log

#####*final*#####`/bin/sh`#####

#####

```
#cloud-config
write_files:
- path: /etc/motd
  content: |
    Welcome to the OpenShift Virtualization VM.
    Managed by cloud-init.
  permissions: '0644'
- path: /etc/myapp/config.yaml
  content: |
    database: postgres
    port: 5432
  owner: root:root
  permissions: '0600'
```

###:

`write\_files`#####*config*#####`runcmd`#####
#####`runcmd`#####

VirtualMachine#####

VirtualMachine#####2#####

#####**cloudInitNoCloud**

#####VirtualMachine#####

#####**VM**#####

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: fedora-cloud-init-demo
  namespace: demo
spec:
  runStrategy: Always
  template:
    metadata:
      labels:
        app: fedora-cloud-init-demo
    spec:
      domain:
        devices:
```

```

disks:
  - disk:
      bus: virtio
      name: rootdisk
  - disk:
      bus: virtio
      name: cloudinitdisk
interfaces:
  - masquerade: {}
    name: default
resources:
  requests:
    memory: 2Gi
networks:
  - name: default
    pod: {}
volumes:
  - dataVolume:
      name: fedora-cloud-init-demo
      name: rootdisk
  - cloudInitNoCloud: ①
    userData: | ②
      #cloud-config
      ssh_pwauth: true
      user:
        name: fedora
        lock_passwd: false
        plain_text_passwd: s3cur3p4ss
        ssh_authorized_keys:
          - 'ssh-ed25519 AAAA...your-key... user@workstation'
    users:
      - name: userone
        lock_passwd: false
        plain_text_passwd: s3cur3p4ss
        ssh_authorized_keys:
          - 'ssh-ed25519 AAAA...your-key... user@workstation'
    packages:
      - vim
      - git
    runcmd:
      - echo "Cloud-init complete" > /tmp/cloud-init-done
      name: cloudinitdisk
dataVolumeTemplates:
  - metadata:
      name: fedora-cloud-init-demo
    spec:
      sourceRef:
        kind: DataSource
        name: fedora
        namespace: openshift-virtualization-os-images
      storage:

```

```
resources:
  requests:
    storage: 30Gi
```

① `cloudInitNoCloud` NoCloud

② `userData` #cloud-config cloud-configYAML

Secret

Secret

Kubernetes Secret VirtualMachine

cloud-init-secret-vm.yaml

Secret:

```
oc create secret generic my-vm-cloud-init \
  --from-literal=userdata='#cloud-config
ssh_pwauth: true
user:
  name: fedora
  lock_passwd: false
  plain_text_passwd: s3cur3p4ss
  ssh_authorized_keys:
    - ssh-ed25519 AAAA...your-key... user@workstation
users:
  - name: userone
    lock_passwd: false
    plain_text_passwd: s3cur3p4ss
    ssh_authorized_keys:
      - ssh-ed25519 AAAA...your-key... user@workstation
packages:
  - vim
  - git
' \
-n demo
```

VirtualMachine:

```
volumes:
  - cloudInitNoCloud:
      secretRef:
        name: my-vm-cloud-init ①
      name: cloudinitdisk
```

① KubeVirt Secret `userdata` userData

VM에서 cloud-init을 실행하는 방법

VM에서 cloud-init을 실행하는 방법

VM에서 cloud-init을 실행하는 방법:

```
virtctl console fedora-cloud-init-demo -n demo
```

cloud-init의 실행 상태를 확인하는 방법:

```
cloud-init status --long
```

cloud-init의 실행 상태를 확인하는 방법:

```
status: done
extended_status: done
boot_status_code: enabled-by-generator
detail:
DataSourceNoCloud [seed=/dev/vdb]
```

cloud-init의 실행 로그를 확인하는 방법:

```
cat /var/log/cloud-init.log | tail -50
```

cloud-init의 실행 로그를 확인하는 방법: **cloud-init.log** 파일을 확인합니다.

cloud-init의 실행 로그를 확인하는 방법:

- **#cloud-config**의 **cloud-init** 섹션에서 **cloud-init**의 실행 상태를 확인합니다. **cloud-init**의 실행 상태를 확인하는 방법은 **cloud-init**의 실행 로그를 확인하는 방법과 동일합니다.

YAML의 실행 방법

cloud-init의 실행 방법을 확인하는 방법은 **cloud-init**의 실행 로그를 확인하는 방법과 동일합니다. **cloud-init**의 실행 방법을 확인하는 방법은 **cloud-init**의 실행 로그를 확인하는 방법과 동일합니다.

cloud-init의 실행 방법을 확인하는 방법: **cloud-init**의 실행 방법을 확인하는 방법

cloud-init의 실행 방법을 확인하는 방법: **cloud-init**의 실행 방법을 확인하는 방법. **cloud-init**의 실행 방법을 확인하는 방법은 **cloud-init**의 실행 로그를 확인하는 방법과 동일합니다.

cloud-init의 실행 방법을 확인하는 방법: **cloud-init**의 실행 방법을 확인하는 방법

cloud-init의 실행 방법을 확인하는 방법: **cloud-init**의 실행 방법을 확인하는 방법. **cloud-init**의 실행 방법을 확인하는 방법은 **cloud-init**의 실행 로그를 확인하는 방법과 동일합니다.

Secret의 실행 방법

secretRef의 실행 방법을 확인하는 방법은 **secretRef**의 실행 방법을 확인하는 방법과 동일합니다.

data`

Windows VM: Sysprep

Linux VM OpenShift Virtualization Microsoft System Preparation Sysprep

Windows VM Windows VM

:

```
oc delete vm fedora-cloud-init-demo -n demo
oc delete vm fedora-cloud-init-secret-demo -n demo
oc delete secret my-vm-cloud-init -n demo
oc delete namespace demo
```

:

- Linux VM
- KubeVirt NoCloud VM
- 3 userData cloud-config networkData network-config v2 metadata
-
- cloudInitNoCloud Secret
- cloud-init status --long`/var/log/cloud-init.log`

- IP - cloud-init IP DHCP VLAN
- IP - cloud-init
- Windows 11 VM Red Hat OpenShift RDP - Sysprep Windows VM
- window=_blank
- window=_blank
- KubeVirt cloudInitNoCloud window=_blank

OpenShift Virtualization

Overview

OpenShift Virtualization uses OpenShift LVM Storage Operator and HostPath Provisioner to manage virtual machines (VMs).

VMs are managed by the OpenShift Virtualization operator.

Architecture

The architecture consists of the following components:

- **LVM Storage Operator** manages the underlying storage infrastructure.
- **HostPath Provisioner** manages the virtual machine disks.
- **OpenShift Virtualization** manages the virtual machines.

OpenShift Virtualization Architecture

OpenShift Virtualization uses the following components:

- **LVM Storage:** Logical Volume Manager (LVM) storage.
- **HostPath Provisioner:** HostPath Provisioner (HostPath Provisioner).
- **Ceph/ODF:** Ceph/ODF storage.
- **NFS:** NFS storage.
- **Cloud Providers:** AWS EBS, Azure Disk, etc.

OpenShift Virtualization uses LVM Storage and HostPath Provisioner to manage virtual machines (VMs) in a Single Node OpenShift (SNO) environment.

Prerequisites

The following prerequisites must be met:

- OpenShift 4.18+ is required.
- A Linux operating system is required.
- At least 1 GB of memory is required.
- Linux kernel (LVM) is required.

目 次

LVM Operator について

OpenShift LVM Storage Operator について

[storage:lvn-troubleshooting]

LVM について

[storage:hostpath-provisioner]

HostPath Provisioner について

[storage:multi-disk-configuration]

StorageClasses について VM について

OpenShift Virtualization について

OpenShift Virtualization について

- DataSource OS について
- VM について
- VM について
- VM について

LVM operator について 5: について

目 次

OpenShift LVM Storage Operator について

- OpenShift VM について
- OpenShift VM について

目 次

- OpenShift LVM Storage について
- OpenShift LVM Storage について

LVM Operator について

目 次

OpenShift LVM Storage operator について LVM Storage operator について Logical Volume Manager について Single Node OpenShift (SNO) について


```
root:
```

```
PV VG Fmt Attr PSize PFree /dev/sda vg1 lvm2 a-- 558.37g <55.84g
```

```
cat </dev/sda> | dd of=/dev/zero bs=1M count=1024
forceWipeDevicesAndDestroyAllData: true`
[source,bash,role=execute]
```

```
oc debug node/<node-name> -- chroot /host ls -la /dev/disk/by-path/ | grep -E "pci.*scsi"
```

```
root:
```

```
lrwxrwxrwx. 1 root root 9 Oct 6 13:30 pci-0000:03:00.0-scsi-0:2:1:0 -> ../../sda
```

```
cat </dev/disk/by-path/pci-0000:03:00.0-scsi-0:2:1:0> | dd of=/dev/zero bs=1M count=1024
== 2: LVM Storage Operator
LVM operator
LVM operator:
xref:attachment$lvm-operator/lvm-namespace.yaml[lvm-namespace.yaml]
[source,bash,role=execute]
```

```
oc apply -f - <<EOF apiVersion: v1 kind: Namespace metadata: name: openshift-lvm-storage EOF
```

```
OperatorGroup:
xref:attachment$lvm-operator/lvm-operatorgroup.yaml[lvm-operatorgroup.yaml]
[source,bash,role=execute]
```

```
oc apply -f - <<EOF apiVersion: operators.coreos.com/v1 kind: OperatorGroup metadata: name:
openshift-lvm-storage-operatorgroup namespace: openshift-lvm-storage spec: targetNamespaces: -
openshift-lvm-storage EOF
```

```

xref:attachment$lvm-operator/lvm-subscription.yaml[lvm-subscription.yaml]
```

```
[source,bash,role=execute]
```

```
oc apply -f - <<EOF apiVersion: operators.coreos.com/v1alpha1 kind: Subscription metadata: name: lvms-operator namespace: openshift-lvm-storage spec: channel: stable-4.20 name: lvms-operator source: redhat-operators sourceNamespace: openshift-marketplace installPlanApproval: Automatic EOF
```

```
#####:
```

```
[source,bash,role=execute]
```

(2-3 #####)

```
oc get csv -n openshift-lvm-storage
```

```
#####
```

```
oc get pods -n openshift-lvm-storage
```

```
##### (LVMCluster #####):
```

```
NAME READY STATUS RESTARTS AGE lvms-operator-864b6975bb-kxt4t 1/1 Running 0 2m
```

```
0: #####`lvms-operator` #####`vg-manager` ##### 3  LVMCluster #####
```

```
##### (#### 3  LVMCluster #####):
```

```
NAME READY STATUS RESTARTS AGE lvms-operator-864b6975bb-kxt4t 1/1 Running 0 5m vg-manager-xxxxxx 1/1 Running 0 1m
```

```
== #### 3: LVMCluster #####
```

```
LVM ##### LVMCluster #####`forceWipeDevicesAndDestroyAllData`  
#####
```

```
=== #####
```

```
##### (0: `sda` 0) #####`forceWipeDevicesAndDestroyAllData: true` #####:
```

```

❶: 'forceWipeDevicesAndDestroyAllData: true'
=====

xref:attachment$lvm-operator/lvmcluster-with-wipe.yaml[lvmcluster-with-wipe.yaml
=====]

[source,bash,role=execute]

```

```

oc apply -f - <<EOF apiVersion: lvm.topolvm.io/v1alpha1 kind: LVMCluster metadata: name:
lvmcluster namespace: openshift-lvm-storage spec: storage: deviceClasses: - name: vg1 default: true
thinPoolConfig: name: thin-pool-1 sizePercent: 90 overprovisionRatio: 10 nodeSelector:
nodeSelectorTerms: - matchExpressions: - key: node-role.kubernetes.io/worker operator: In values:
- "" deviceSelector: paths: - /dev/sda forceWipeDevicesAndDestroyAllData: true EOF

```

```

=== =====

=====:

xref:attachment$lvm-operator/lvmcluster-clean-disk.yaml[lvmcluster-clean-disk.yaml
=====]

[source,bash,role=execute]

```

```

oc apply -f - <<EOF apiVersion: lvm.topolvm.io/v1alpha1 kind: LVMCluster metadata: name:
lvmcluster namespace: openshift-lvm-storage spec: storage: deviceClasses: - name: vg1 default: true
thinPoolConfig: name: thin-pool-1 sizePercent: 90 overprovisionRatio: 10 nodeSelector:
nodeSelectorTerms: - matchExpressions: - key: node-role.kubernetes.io/worker operator: In values:
- "" deviceSelector: paths: - /dev/sda EOF

```

```

=== =====

* **`name: vg1`**: ===== (=====)
* **`default: true`**: =====
**  : =====
**  : ===== StorageClass  PVC =====
**  : =====1=====
* **`thinPoolConfig`**: =====
** `sizePercent: 90`: ===== 90%
** `overprovisionRatio: 10`: ===== 10 =====
* **`nodeSelector`**: ===== (=====)
* **`deviceSelector.paths`**: =====
** `/dev/sda` `/dev/disk/by-path/...` `/dev/disk/by-id/...` =====
* **`forceWipeDevicesAndDestroyAllData: true`**: =====
**  : =====
**  : ===== LVM
**  : ===== -
**  : =====

```



```
== ステップ 4: LVMCluster のインストール

LVMCluster のインストールコマンドを実行:

[source,bash,role=execute]
```

LVMCluster のインストール

```
oc get lvmcluster -n openshift-lvm-storage
```

```
lvmcluster.openshift-lvm-storage 1/1 Running 0s
```

```
oc debug node/<node-name> -- chroot /host vgs
```

```
lvmcluster.openshift-lvm-storage 1/1 Running 0s
```

```
oc debug node/<node-name> -- chroot /host lvs
```

```
root@node1:~#

* LVMCluster の 'STATUS: Ready' の確認
* VG 'vg1' の確認
* 'thin-pool-1' の確認/リサイズ

xref:lvm-troubleshooting.adoc[LVM Storage Troubleshooting Guide]

[[_step_5_set_default_storageclass]]
== ステップ 5: デフォルト StorageClass の設定

LVM operator の 'lvms-vg1' のデフォルト StorageClass の設定
Virtualization の DataVolume の StorageClass の設定

=== デフォルト StorageClass の確認

[source,bash,role=execute]
```

```
oc get storageclass
```

```
root@node1:~#
```

```
NAME PROVISIONER RECLAIMPOLICY VOLUMEBINDINGMODE ALLOWVOLUMEEXPANSION AGE
lvms-vg1 topolvm.io Delete WaitForFirstConsumer true 5m
```



```
[source,yaml]
```

```
dataVolumeTemplates: - metadata: name: fedora-volume spec: pvc: accessModes: - ReadWriteOnce
resources: requests: storage: 30Gi sourceRef: kind: DataSource name: fedora namespace: openshift-
virtualization-os-images
```

```
##### StorageClass ('lvms-vg1') #####

==== VM

##### StorageClass #####:

* #####
* DataSource #####
* PVC  'storageClassName' #####
* #####

====

##### StorageClass #####OpenShift Virtualization ##### OS #####:

[source,bash,role=execute]
```

#####

```
oc get DataImportCron -n openshift-virtualization-os-images
```

```
=== VM ##### StorageClass

##### StorageClass ##### VM #####:

[source,yaml]
```

```
dataVolumeTemplates: - metadata: name: fedora-volume spec: pvc: accessModes: - ReadWriteOnce
resources: requests: storage: 30Gi storageClassName: lvms-vg1 # ##### sourceRef: kind: DataSource
name: fedora namespace: openshift-virtualization-os-images
```

```
VM: OpenShift Virtualization ##### StorageClass #####

== VM 6: #####

PVC #####:

xref:attachment$lvms-operator/test-pvc.yaml[test-pvc.yaml #####]
```

```
[source,bash,role=execute]
```

```
oc apply -f - <<EOF apiVersion: v1 kind: PersistentVolumeClaim metadata: name: lvms-test-pvc
namespace: default spec: accessModes: - ReadWriteOnce resources: requests: storage: 10Gi
storageClassName: lvms-vg1 EOF
```

```
#####:
```

```
xref:attachment$lvms-operator/test-pod.yaml[test-pod.yaml #####]
```

```
[source,bash,role=execute]
```

```
oc apply -f - <<EOF apiVersion: v1 kind: Pod metadata: name: lvms-test-pod namespace: default
spec: containers: - name: test image: registry.access.redhat.com/ubi9/ubi:latest command: ["sleep",
"3600"] volumeMounts: - name: data mountPath: /data volumes: - name: data
persistentVolumeClaim: claimName: lvms-test-pvc EOF
```

```
PVC #####:
```

```
[source,bash,role=execute]
```

```
oc get pvc lvms-test-pvc
```

"Bound" STATUS

```
#####:
```

```
[source,bash,role=execute]
```

```
#####
```

```
oc wait --for=condition=Ready pod/lvms-test-pod --timeout=120s
```

```
#####
```

```
oc exec lvms-test-pod -- sh -c "echo 'LVM Storage Test' > /data/test.txt && cat /data/test.txt"
```

#####: LVM Storage Test

```
#####:
```

```
[source,bash,role=execute]
```

oc delete pod lvms-test-pod oc delete pvc lvms-test-pvc

```
==  {{{{
```

```
LVM  {{{{xref:lvn-troubleshooting.adoc[LVM Storage Troubleshooting Guide]  {{{{
```

```
~~~~~
```

```
*  {{{{
*  {{{{
*  {{{{
*  PVC  {{{{
*  {{{{
*  {{{{
```

```
== forceWipeDevicesAndDestroyAllData  {{{
```

```
===  {{{
```

```
`forceWipeDevicesAndDestroyAllData: true`  {{{:
```

```
.  **{{{**: GPTMBR{{{
.  **{{{**: ext4xfsbtrfs{{{
.  **LVM  {{{**:  {{{
.  **{{{**:  {{{
.  **{{{**: LVM operator  {{{
```

```
===  {{{
```

```
`forceWipeDevicesAndDestroyAllData: true`  {{{:
```

```
*  {{{
*  {{{ LVM (PV/VG/LV  {{{)  {{{
*  {{{
*  {{{
```

```
~~~~~:
```

```
*  {{{
*  {{{ ({{{!))
*  {{{
```

```
===  {{{
```

```
~~~~~`forceWipeDevicesAndDestroyAllData: true`  {{{:
```

1. 检查块设备

oc debug node/<node-name> — chroot /host lsblk -o NAME,SIZE,TYPE,MOUNTPOINT

2. 检查挂载点

oc debug node/<node-name> — chroot /host mount | grep /dev/sda

3. 检查磁盘空间

oc debug node/<node-name> — chroot /host df -h | grep sda

4. 检查设备名称和序列号

oc debug node/<node-name> — chroot /host lsblk -d -o NAME,SERIAL /dev/sda

== 检查

检查块设备:

```
* 检查块设备 LVM 检查块设备
* LVM Storage operator 检查块设备OperatorGroup检查块设备
* 'forceWipeDevicesAndDestroyAllData' 检查块设备 LVMCluster 检查块设备
* 检查块设备 'lvms-vg1' StorageClass 检查块设备 OpenShift Virtualization 检查块设备 StorageClass 检查块设备
* PVC 检查块设备 Pod 检查块设备 LVMCluster 检查块设备
```

== 检查

*

link:https://docs.redhat.com/en/documentation/openshift_container_platform/4.20/html/storage/persistent-storage-using-local-storage#persistent-storage-using-lvms[OpenShift Container Platform 4.20 检查块设备,window=_blank]

*

link:https://docs.redhat.com/en/documentation/openshift_container_platform/4.20/html/storage/understanding-persistent-storage#storage-classes_understanding-persistent-storage[OpenShift Container Platform 4.20 检查块设备,window=_blank]

:!navtitle:

:docname: lvm-troubleshooting

:page-module: storage

:page-relative-src-path: lvm-troubleshooting.adoc

```

:page-origin-url: https://github.com/RedHatQuickCourses/ocp-virt-cookbook_ja
:page-origin-start-path:
:page-origin-refname: main
:page-origin-reftype: branch
:page-origin-refhash: (worktree)
:navtitle: LVM
[#storage:lvm-troubleshooting]
=== LVM

[discrete#storage:lvm-troubleshooting::_]
===

OpenShift LVM LVM
LVM

[discrete#storage:lvm-troubleshooting::_]
===

LVM

[source,bash,role=execute]

```

1. LVMCluster

```
oc get lvmcluster -n openshift-lvm-storage
```


2. □□□□□□□□□□□□□□□□

```
oc get pods -n openshift-lvm-storage
```

3. StorageClass□□□□□□□□□□

oc get storageclass | grep lvms

4. □□□□□□□□□□□□□□□□

oc debug node/<node-name> — chroot /host vgs

5. 检查存储状态

oc debug node/<node-name> — chroot /host lvs

```
root@node1:~# lvs

* LVMCluster: `STATUS: Ready`
* 节点: 节点`Running`
* StorageClass: `lvs-vg1`
* VG: 节点/节点
* LV: 节点/节点

[discrete#storage:lvm-troubleshooting::_节点]
==== 节点

[discrete#storage:lvm-troubleshooting::_lvmcluster节点]
===== LVMCluster节点

[discrete#storage:lvm-troubleshooting::_节点]
===== 节点

[source,bash,role=execute]
```

oc get lvmcluster -n openshift-lvm-storage

```
root@node1:~#
```

NAME STATUS lvmcluster Ready

```
root@node1:~# lvs

* 节点`NotReady` `Degraded` `Error`
* 节点

[discrete#storage:lvm-troubleshooting::_节点]
===== 节点

[source,bash,role=execute]
```

oc get lvmcluster lvmcluster -n openshift-lvm-storage -o yaml

```
root@node1:~#

[source,yaml]
```

status: conditions: - status: "True" type: ResourcesAvailable message: Reconciliation is complete and all the resources are available - status: "True" type: VolumeGroupsReady message: All the VGs are ready deviceClassStatuses: - name: vg1 nodeStatus: - devices: - /dev/sda status: Ready ready: true state: Ready

```
root@node1:~# oc get lvmcluster -n openshift-lvm-storage -o yaml | grep -A 50 "deviceClassStatuses"

* `status.conditions`: `ResourcesAvailable: True` `VolumeGroupsReady: True`
* `deviceClassStatuses[].nodeStatus[].status`: `Ready`
* `deviceClassStatuses[].nodeStatus[].devices`: 
* `ready: true` `state: Ready`

[discrete#storage:lvm-troubleshooting::_]
=====

[source,bash,role=execute]
```

oc get lvmcluster lvmcluster -n openshift-lvm-storage -o yaml | grep -A 50 "deviceClassStatuses"

```
root@node1:~# oc get lvmcluster -n openshift-lvm-storage -o yaml | grep -A 50 "deviceClassStatuses"

[source,yaml]
```

deviceClassStatuses: - name: vg1 nodeStatus: - devices: - /dev/sda excluded: - name: /dev/sdc reasons: - /dev/sdc status: Ready

```
root@node1:~# oc get lvmcluster -n openshift-lvm-storage -o yaml | grep -A 50 "deviceClassStatuses"

[discrete#storage:lvm-troubleshooting::_]
=====

[discrete#storage:lvm-troubleshooting::_]
=====

[source,bash,role=execute]
```

oc get pods -n openshift-lvm-storage

```
root@node1:~# oc get pods -n openshift-lvm-storage
```

NAME READY STATUS RESTARTS AGE lvms-operator-864b6975bb-kxt4t 1/1 Running 0 10m vg-manager-xxxxxx 1/1 Running 0 5m

```
root@node1:~# oc get pods -n openshift-lvm-storage
```

```
* **lvm-operator**: 管理LVM
* **vg-manager**: 管理1个VG
* **topolvm-controller**: 管理OpenShift (SNO)的LVM
```

```
[discrete#storage:lvm-troubleshooting::_管理LVM]
```

```
===== 管理LVM
```

```
[source,bash,role=execute]
```



oc describe pod -n openshift-lvm-storage <pod-name>

□ □ □ □ □ □ □ □ □ □ □

oc logs -n openshift-lvm-storage <pod-name>

vg-manager

()

oc logs -n openshift-lvm-storage daemonset/vg-manager

:

```
* `device is in use` - 
* `cannot wipe device` - `forceWipeDevicesAndDestroyAllData: true`
* `device not found` - 
* `no devices found` - 
```

```
[discrete#storage:lvm-troubleshooting::_]
===== 
```

```
[source,bash,role=execute]
```

ClusterServiceVersion□□□

oc get csv -n openshift-lvm-storage



oc get subscription -n openshift-lvm-storage

□□□□□□□□□□□□

oc get installplan -n openshift-lvm-storage

```
[discrete#storage:lvm-troubleshooting::_□□□□□□□□]
```

```
===== □□□□□□□□
```

```
[discrete#storage:lvm-troubleshooting::_storageclass□□□]
```

```
===== StorageClass□□□
```

```
[source,bash,role=execute]
```

StorageClass

oc get storageclass

■■■LVM■■■■■StorageClass■■■

oc get storageclass lvms-vg1 -o yaml

■■■■:

NAME PROVISIONER RECLAIMPOLICY VOLUMEBINDINGMODE ALLOWVOLUMEEXPANSION AGE
lvms-vg1 topolvm.io Delete WaitForFirstConsumer true 10m

■■■■■■■■:

- * `PROVISIONER: topolvm.io` - ■■■■■■■■
- * `VOLUMEBINDINGMODE: WaitForFirstConsumer` - ■■■■■■■■
- * `ALLOWVOLUMEEXPANSION: true` - ■■■■■■■■

[discrete#storage:lvm-troubleshooting::_storageclass■■■■■■■■■■■■■■■■■■■■]
===== StorageClass■■■■■■■■■■■■■■■■■■■■

[source,bash,role=execute]

oc get storageclass lvms-vg1 -o jsonpath='{.metadata.annotations.storageclass\.kubernetes\.io/is-default-class}'

■■:

- * `true` - ■■■■■■■■StorageClass
- * ■■■■■■■■`false` - ■■■■■■■■

[discrete#storage:lvm-troubleshooting::_persistentvolumes■■■]
===== PersistentVolumes■■■

[source,bash,role=execute]

□□□□**PV**□□□□□□

oc get pv

LVM PV

oc get pv | grep topolvm

```
[discrete#storage:lvm-troubleshooting::_]
=====

[discrete#storage:lvm-troubleshooting::_]
=====

[source,bash,role=execute]
```

oc debug node/<node-name> — chroot /host vgs

```
root:
```

VG #PV #LV #SN Attr VSize VFree vg1 1 1 0 wz--n- 558.37g <55.84g

```
root:

* VG LVMCluster ([: `vg1`)
* `VSize`
* `VFree`
* `Attr` `wz--n-` ( )
[discrete#storage:lvm-troubleshooting::_]
=====

[source,bash,role=execute]
```

oc debug node/<node-name> — chroot /host vgdisplay vg1

```
[discrete#storage:lvm-troubleshooting::_]
root:
```

--- VG Name vg1 System ID Format lvm2 Metadata Areas 1 Metadata Sequence No 2 VG Access read/write VG Status resizable MAX LV 0 Cur LV 1 Open LV 0 Max PV 0 Cur PV 1 Act PV 1 VG Size 558.37 GiB PE Size 4.00 MiB Total PE 142942 Alloc PE / Size 128517 / 502.04 GiB Free PE / Size 14425 / 56.35 GiB VG UUID abc123...

```
root:

* `VG Status: resizable`
* `VG Access: read/write`
```



```
* `Free PE / Size`
```

```
===
```

```
====
```

```
[source,bash,role=execute]
```

oc debug node/<node-name> — chroot /host lvs

```
:
```

LV VG Attr LSize Pool Origin Data% Meta% thin-pool-1 vg1 twi-a-tz-- <502.04g 0.00 6.76

```
(twi-a-tz--):
```

```
* `t` -
```

```
* `w` -
```

```
* `i` -
```

```
* `a` -
```

```
* `t` -
```

```
* `z` -
```

```
%:
```

```
* **%*:
```

```
** `< 80%` -
```

```
** `80-90%` -
```

```
** `> 90%` -
```

```
* **Meta%*:
```

```
** `< 80%` -
```

```
** `> 80%` -
```

```
====
```

```
[source,bash,role=execute]
```

oc debug node/<node-name> — chroot /host lvdisplay vg1/thin-pool-1

```
===
```

```
====
```

```
[source,bash,role=execute]
```

oc debug node/<node-name> — chroot /host pvs

```
#####:
```

```
PV VG Fmt Attr PSize PFree /dev/sda vg1 lvm2 a-- 558.37g <55.84g
```

```
##### (a--):
```

```
* `a` -  ##
* `-' - #####
* `-' - #####
```

```
==== #####
```

```
[source,bash,role=execute]
```

```
oc debug node/<node-name> — chroot /host pvdisplay /dev/sda
```

```
==== #####
```

```
[source,bash,role=execute]
```

```
#####
```

```
oc debug node/<node-name> — chroot /host ls -la /dev/sda
```

```
#####
```

```
oc debug node/<node-name> — chroot /host lsblk /dev/sda
```

```
#####
```

```
oc debug node/<node-name> — chroot /host mount | grep /dev/sda
```

```
␣: #####
```

```
=== #####
```

```
==== #####
```

```
[source,bash,role=execute]
```

```
oc          debug          node/<node-name> — chroot          /host          lsblk          -o
NAME,SIZE,TYPE,FSTYPE,MOUNTPOINT,MODEL
```

```
root:
```

```
NAME SIZE TYPE FSTYPE MOUNTPOINT MODEL sda 558.4G disk LVM2_member PERC H730 Adp
sdc 558.4G disk PERC H730 Adp |-sdc1 1M part |-sdc2 127M part vfat |-sdc3 384M part ext4 /boot `
sdc4 557.9G part xfs /sysroot
```

```
root@node1:~#
```

```
* `MOUNTPOINT` - 物理磁盘挂载点
* `FSTYPE` - 文件系统类型 (LVM2_member) - 逻辑卷所属的存储池
* `MOUNTPOINT` - 物理磁盘挂载点
* 物理磁盘 - 物理磁盘
```

```
==== 物理磁盘
```

```
[source,bash,role=execute]
```

oc debug node/<node-name> — chroot /host lsblk -d -o NAME,SIZE,SERIAL

```
root@node1:~#
```

```
==== 物理磁盘
```

```
[source,bash,role=execute]
```

物理磁盘

oc debug node/<node-name> — chroot /host ls -la /dev/disk/by-path/ | grep sda

ID物理磁盘

oc debug node/<node-name> — chroot /host ls -la /dev/disk/by-id/ | grep sda

```
== 物理磁盘
```

```
=== 01: LVMCluster "NotReady" 物理磁盘
```

```
root:
```

NAME STATUS lvmcluster NotReady

```
root:
```

```
[source,bash,role=execute]
```

```
oc get lvmcluster lvmcluster -n openshift-lvm-storage -o yaml | grep -A 20 "conditions"
```

vg-manager

```
oc logs -n openshift-lvm-storage daemonset/vg-manager
```

```
vg-manager:
```

```
====  
```

```
[source,bash,role=execute]
```

```
oc debug node/<node-name> -- chroot /host ls -la /dev/sda
```

```
root: LVMCluster
```

```
====  /
```

```
[source,bash,role=execute]
```

```
oc debug node/<node-name> -- chroot /host lsblk -f /dev/sda
```

```
root: `forceWipeDevicesAndDestroyAllData: true` `deviceSelector`
```

```
====  
```

```
[source,bash,role=execute]
```

oc debug node/<node-name> — chroot /host mount | grep /dev/sda

oc debug node/<node-name> — chroot /host mount | grep /dev/sda

```
root: ~
=== root: ~

root:

* LVMCluster`vgs`
* `deviceClassStatuses`

root:

[source,bash,role=execute]
```

oc debug node/<node-name> — chroot /host pvs

oc debug node/<node-name> — chroot /host pvs

vg-manager

oc logs -n openshift-lvm-storage daemonset/vg-manager --tail=100

oc debug node/<node-name> — chroot /host lsblk -o NAME,SIZE,TYPE,FSTYPE

oc debug node/<node-name> — chroot /host lsblk -o NAME,SIZE,TYPE,FSTYPE

```
root:

====

* `deviceSelector`
* (see `deviceClassStatuses.excluded`)

====

* vg-manager
* SELinux

====

* `forceWipeDevicesAndDestroyAllData: true`
```



```
==== 0000000000
```

```
[source,bash,role=execute]
```

00000000000000000000

```
oc debug node/<node-name> — chroot /host lvs | grep thin-pool
```

```
000: 0000000000000000000000000000000000
```

```
==== 0000000000000000
```

```
[source,bash,role=execute]
```

00000000000000000000LVMCluster0000000000

0000000

```
oc get pod <pod-name> -o yaml | grep nodeSelector oc get lvmcluster lvmcluster -n openshift-lvm-storage -o yaml | grep -A 5 nodeSelector
```

```
=== 004: 0000000000%00000000%000
```

```
00:
```

```
LV VG Attr LSize Data% Meta% thin-pool-1 vg1 twi-a-tz-- 502.04g 92.00 85.00
```

```
00:
```

```
[source,bash,role=execute]
```

00000000000000000000

```
oc debug node/<node-name> — chroot /host lvs -a | grep thin-pool
```

0000000000000000000000000000000000

```
oc debug node/<node-name> — chroot /host lvs -a -o lv_name,lv_size,data_percent,pool_lv | grep thin-pool
```

PVC 목록 조회 PV 목록

```
oc get pv -o custom-columns=NAME:.metadata.name,SIZE:.spec.capacity.storage,STORAGECLASS:.spec.storageClassName | grep lvms
```

```
root:

====  PVC 목록

[source,bash,role=execute]
```

모든 PVC 목록

```
oc get pvc --all-namespaces
```

PVC 삭제

```
oc delete pvc <pvc-name> -n <namespace>
```

```
====  VG 목록 (VG 목록 조회)

[source,bash,role=execute]
```

VG 목록 조회

```
oc debug node/<node-name> -- chroot /host vgs
```

모든 VG 목록

(모든 LVM 목록 조회)

```
====  VG 목록

LVMCluster`overprovisionRatio`

===  5: VG 목록

root:
```



```
* LVMCluster
* "cannot wipe device" "device busy"
```

:

```
[source,bash,role=execute]
```

/

```
oc debug node/<node-name> — chroot /host lsblk -f /dev/sda
```

LVM

```
oc debug node/<node-name> — chroot /host pvs | grep /dev/sda
```

```
oc debug node/<node-name> — chroot /host lsof | grep /dev/sda
```

:

```
[source,bash,role=execute]
```

`forceWipeDevicesAndDestroyAllData``

```
oc edit lvmcluster lvmcluster -n openshift-lvm-storage
```

`deviceSelector`

```
forceWipeDevicesAndDestroyAllData: true
```

```
=== 6: vg-manager PodCrashLoopBackOff
```

:

```
NAME READY STATUS RESTARTS AGE vg-manager-xxx 0/1 CrashLoopBackOff 5 3m
```

:

```
[source,bash,role=execute]
```

```
oc logs -n openshift-lvm-storage <vg-manager-pod> --previous
```

```
oc describe pod -n openshift-lvm-storage <vg-manager-pod>
```

```
oc describe node <node-name>
```

```
#####:

=====  #####

*  vg-manager#####
*  #####SELinux#####

=====  #####

*  #####
*  #####

=====  #####

*  #####CPU#####
*  #####

===  7: StorageClass#####

###:

*  #####PVC#####
*  #: "no default deviceClass was specified"

###:

[source,bash,role=execute]
```

StorageClass

oc get storageclass | grep default

lvms StorageClass

oc get storageclass lvms-vg1 -o yaml | grep is-default-class

```
---:

==== lvms StorageClass

[source,bash,role=execute]
```

oc patch storageclass lvms-vg1 -p '{"metadata": {"annotations":{"storageclass.kubernetes.io/is-default-class":"true"}}}'

```
==== LVMCluster

[source,bash,role=execute]
```

oc edit lvmcluster lvmcluster -n openshift-lvm-storage

deviceClasses

- name: vg1 default: true

```
==

LVM

[source,bash,role=execute]
```

#!/bin/bash

NAMESPACE="openshift-lvm-storage" NODE_NAME="<your-node-name>"

echo "=== LVM ===" echo ""

echo "1. LVMCluster:" oc get lvmcluster -n \$NAMESPACE echo ""

echo "2. Pods:" oc get pods -n \$NAMESPACE echo ""

echo "3. StorageClass:" oc get storageclass | grep lvms echo ""


```
===== HostPath Provisioner
```

```
HPP
```

```
* ***: :  
* ***: :  
* ***: :  
* ***: :
```

```
[discrete#storage:hostpath-provisioner::_lvm_storage_]
```

```
===== LVM Storage
```

```
[cols="2,2,2"]
```

```
|==
```

```
| |HostPath Provisioner |LVM Storage
```

```
|  
| ( )  
| ( )
```

```
|  
|  
|
```

```
|  
| ( )  
| ( )
```

```
|  
| ( )  
|
```

```
|  
|/  
|
```

```
|==
```

```
xref:storage:lvm-operator[LVM Storage Operator]  

```

```
[discrete#storage:hostpath-provisioner::_]
```

```
=====
```

```
* HostPathProvisioner  
* HPP StorageClass  
* SELinux  
* PVC VM
```

```
:
```

```
[discrete#storage:hostpath-provisioner::_0000]
==== 0000

* **OpenShift 4.21+**  OpenShift Virtualization 000000000000
* **000000000000**: HostPathProvisioner  StorageClass 000000000000
* **00000000000000000000**: 00000000 VM 000000000000000000
* **oc CLI**: 000000000000
* **virtctl CLI**: VM 000000000000000000

OpenShift Virtualization 00000000000000000000:

[source,bash,role=execute]
```

```
oc get csv -n openshift-cnv | grep kubevirt
```

```
0000000 (0000000000):
```

kubevirt-hyperconverged-operator.v4.x.x OpenShift Virtualization 4.x.x Succeeded

```
[discrete#storage:hostpath-provisioner::_0000_1_000000000000]
==== 0000 1: 000000000000

HPP 0000000000000000000000000000000000000000000000000000000 SELinux 000000000000000000000000

[discrete#storage:hostpath-provisioner::_00000_a_machineconfig_00000_0000000]
===== 00000 A: MachineConfig 00000 (00000000)

MachineConfig 0000000000000000000000000000000000000000000000000000000:

link:attachment$hostpath-provisioner/hpp-machineconfig.yaml[hpp-machineconfig.yaml
0000000,role=unresolved]

[source,bash,role=execute]
```

```
oc apply -f - <<EOF apiVersion: machineconfiguration.openshift.io/v1 kind: MachineConfig
metadata: name: 50-hpp-directory labels: machineconfiguration.openshift.io/role: worker spec:
config: ignition: version: 3.2.0 systemd: units: - name: create-hpp-directory.service enabled: true
contents: | [Unit] Description=Create HostPath Provisioner directory Before=kubelet.service
```

```
[Service]
Type=oneshot
ExecStart=/bin/mkdir -p /var/hpvolumes
ExecStart=/bin/chcon -Rt container_file_t /var/hpvolumes
RemainAfterExit=yes
```

```
[Install]
WantedBy=multi-user.target
EOF
```

```
❯ MachineConfig
MachineConfig
[source,bash,role=execute]
```

oc get mcp worker

```
`UPDATED`  `True`  `UPDATING`  `False`
```

NAME CONFIG UPDATED UPDATING DEGRADED MACHINECOUNT READYMACHINECOUNT worker
rendered-worker-abc123 True False False 6 6

```
[discrete#storage:hostpath-provisioner::_b_]
==== B: (
[discrete#storage:hostpath-provisioner::_b_]
[discrete#storage:hostpath-provisioner::_b_]
[source,bash,role=execute]
```

oc get nodes -l node-role.kubernetes.io/worker

```
(`<worker-node-name>`):
[source,bash,role=execute]
```

oc debug node/<worker-node-name> — chroot /host mkdir -p /var/hpvolumes

```
[source,bash,role=execute]
```

oc debug node/<worker-node-name> — chroot /host chcon -Rt container_file_t /var/hpvolumes

```
❯ MachineConfig
[discrete#storage:hostpath-provisioner::_b_]
====
```

```
SELinux permissions:
```

```
[source,bash,role=execute]
```

```
oc debug node/<worker-node-name> -- chroot /host ls -laZ /var/hpvolumes
```

```
permissions:
```

```
total 0 drwxr-xr-x. 2 root root system_u:object_r:container_file_t:s0 6 Mar 19 10:00 . drwxr-xr-x. 22
root root system_u:object_r:var_t:s0 4096 Mar 19 10:00 ..
```

```
'container_file_t' SELinux permissions:
```

```
[discrete#storage:hostpath-provisioner::_hostpathprovisioner_]
==== 2: HostPathProvisioner
```

```
HostPathProvisioner CSI permissions:
```

```
Link:attachment$hostpath-provisioner/hpp-hostpathprovisioner.yaml[hpp-
hostpathprovisioner.yaml ,role=unresolved]
```

```
[source,bash,role=execute]
```

```
oc apply -f - <<EOF apiVersion: hostpathprovisioner.kubevirt.io/v1beta1 kind: HostPathProvisioner
metadata: name: hostpath-provisioner spec: imagePullPolicy: IfNotPresent storagePools: - name:
local path: /var/hpvolumes workload: nodeSelector: kubernetes.io/os: linux EOF
```

```
[discrete#storage:hostpath-provisioner::_]
=====
```

```
* **`storagePools`**: 1
** `name`: StorageClass
** `path`:
* **`workload.nodeSelector`**: HPP CSI
* **`imagePullPolicy`**: 
```

```
[discrete#storage:hostpath-provisioner::_hostpathprovisioner_]
==== HostPathProvisioner
```

```
HostPathProvisioner permissions:
```

```
[source,bash,role=execute]
```

```
oc get hostpathprovisioner hostpath-provisioner
```



```
#####:
```

NAME AGE hostpath-provisioner 30s

```
##### CSI #####:
```

```
[source,bash,role=execute]
```

oc get pods -n openshift-cnv -l k8s-app=hostpath-provisioner

```
##### (##### 1 #####):
```

NAME READY STATUS RESTARTS AGE hostpath-provisioner-csi-xxxxx 4/4 Running 0 2m hostpath-provisioner-csi-yyyyy 4/4 Running 0 2m hostpath-provisioner-csi-zzzzz 4/4 Running 0 2m

```
[discrete#storage:hostpath-provisioner::_3_storageclass_000]
==== 3: StorageClass 000
```

HostPath Provisioner StorageClass:

```
link:attachment$hostpath-provisioner/hpp-storageclass.yaml[hpp-storageclass.yaml
#####,role=unresolved]
```

```
[source,bash,role=execute]
```

oc apply -f - <<EOF apiVersion: storage.k8s.io/v1 kind: StorageClass metadata: name: hostpath-csi provisioner: kubevirt.io.hostpath-provisioner reclaimPolicy: Delete volumeBindingMode: WaitForFirstConsumer parameters: storagePool: local EOF

```
[discrete#storage:hostpath-provisioner::_2]
=====
```

```
* **`provisioner`**: `kubevirt.io.hostpath-provisioner`
* **`reclaimPolicy`**: `Delete` PVC PV `Retain`
* **`volumeBindingMode`**: `WaitForFirstConsumer` PVC PV
#####
* **`parameters.storagePool`**: HostPathProvisioner
```

```
[discrete#storage:hostpath-provisioner::_storageclass_000]
===== StorageClass 000
```

```
[source,bash,role=execute]
```

oc get storageclass hostpath-csi

NAME :

NAME PROVISIONER RECLAIMPOLICY VOLUMEBINDINGMODE ALLOWVOLUMEEXPANSION AGE
hostpath-csi kubevirt.io.hostpath-provisioner Delete WaitForFirstConsumer false 10s

```
[discrete#storage:hostpath-provisioner::_storageclass_]
===== : StorageClass
```

HPP StorageClass (/):

```
[source,bash,role=execute]
```

oc patch storageclass hostpath-csi -p '{"metadata": {"annotations":{"storageclass.kubernetes.io/is-default-class":"true"}}}'

: 1 StorageClass StorageClass :

```
[source,bash,role=execute]
```



```
oc get pvc hpp-test-pvc
```

□□□□□□ :

```
NAME STATUS VOLUME CAPACITY ACCESS MODES STORAGECLASS VOLUMEATTRIBUTESCLASS
AGE hpp-test-pvc Bound pvc-abc12345-6789-def0-1234-567890abcdef 5Gi RWO hostpath-csi <unset>
30s
```

0: HPP

:

```
[source,bash,role=execute]
```

```
oc exec hpp-test-pod — sh -c "echo 'HPP Storage Test' > /data/test.txt && cat /data/test.txt"
```

□□□□□□ :

HPP Storage Test

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

```
[source,bash,role=execute]
```

```
oc delete pod hpp-test-pod oc delete pvc hpp-test-pvc
```

```
[discrete#storage:hostpath-provisioner:::_0000_5_vm_00000]
```

==== 0000 5: VM 00000

HPP VM :

```
00: 0000 cloud-init 000000000000000000000000000000000000 SSH 000000000000
```

```
link:attachment$hostpath-provisioner/hpp-test-vm.yaml[hpp-test-vm.yaml
#####,role=unresolved]
```

```
[source,bash,role=execute]
```

```
oc apply -f - <<EOF apiVersion: kubevirt.io/v1 kind: VirtualMachine metadata: name: hpp-test-vm
namespace: default spec: runStrategy: Always template: metadata: labels: kubevirt.io/vm: hpp-test-
vm spec: domain: cpu: cores: 1 devices: disks: - name: rootdisk disk: bus: virtio - name: cloudinitdisk
disk: bus: virtio interfaces: - name: default masquerade: {} resources: requests: memory: 1Gi
networks: - name: default pod: {} volumes: - name: rootdisk dataVolume: name: hpp-test-vm-
rootdisk - name: cloudinitdisk cloudInitNoCloud: userData: | #cloud-config user: fedora password:
```

fedora chpasswd: expire: false dataVolumeTemplates: - metadata: name: hpp-test-vm-rootdisk spec:
storage: accessModes: - ReadWriteOnce resources: requests: storage: 10Gi storageClassName:
hostpath-csi source: registry: url: docker://quay.io/containerdisks/fedora:latest EOF

DataVolume `test-volume`:

[source,bash,role=execute]

DataVolume

```
oc get dv -n default hpp-test-vm-rootdisk -w
```

```
NAME :
```

```
NAME PHASE PROGRESS RESTARTS AGE hpp-test-vm-rootdisk ImportScheduled N/A 5s hpp-test-vm-rootdisk ImportInProgress 0.00% 10s hpp-test-vm-rootdisk ImportInProgress 45.50% 30s hpp-test-vm-rootdisk Succeeded 100.0% 1m
```

```
VM :
```

```
[source,bash,role=execute]
```

```
oc get vm hpp-test-vm oc get vmi hpp-test-vm
```

```
NAME :
```

```
NAME AGE STATUS READY hpp-test-vm 2m Running True
```

```
NAME AGE PHASE IP NODENAME READY hpp-test-vm 2m Running 10.x.x.x workerXX True
```

```
VM :
```

```
[source,bash,role=execute]
```

```
virtctl console hpp-test-vm
```

```
VM :
```

```
[source,bash,role=execute]
```

```
oc delete vm hpp-test-vm
```

```
[discrete#storage:hostpath-provisioner::_pvc_0_pending_000]
```

```
==== 000000000000
```

```
[discrete#storage:hostpath-provisioner::_pvc_0_pending_000]
```

```
===== PVC 0 Pending 000
```

```
00000000 PVC 0 Pending 000000:
```

[source,bash,role=execute]

PVC □□□□□□□□

oc describe pvc <pvc-name>

HPP Pod 启动失败

oc logs -n openshift-cnv -l k8s-app=hostpath-provisioner -c hostpath-provisioner

podname:

```
* **podname***: podname `/var/hpvolumes` podname
* **SELinux podname***: `chcon -Rt container_file_t /var/hpvolumes` podname
* **HPP Pod podname***: podname HPP CSI Pod podname
```

```
[discrete#storage:hostpath-provisioner::_podname]
===== podname
```

podname:

```
[source,bash,role=execute]
```

□□□□□ **SELinux** □□□□□□□□□□

oc debug node/<worker-node-name> — chroot /host ls -laZ /var/hpvolumes

SELinux 配置

```
oc debug node/<worker-node-name> — chroot /host chcon -Rt container_file_t /var/hpvolumes
```

```
[discrete#storage:hostpath-provisioner::_hpp_pod_000000]
```

```
===== HPP Pod 000000
```

```
HostPathProvisioner 0000000000000000:
```

```
[source,bash,role=execute]
```

```
oc describe hostpathprovisioner hostpath-provisioner
```


Pod 配置 nodeSelector 配置

spec: nodeSelector: kubernetes.io/hostname: <worker-node-name>

```
[discrete#storage:hostpath-provisioner::_[redacted]]
```

```
==== [redacted]
```

HPP [redacted]:

```
[source,bash,role=execute]
```

StorageClass □□□

oc delete storageclass hostpath-csi

HostPathProvisioner ☐☐☐

oc delete hostpathprovisioner hostpath-provisioner

Virtualization (CNV)  OpenShift

- []

API](https://docs.redhat.com/en/documentation/openshift_container_platform/4.20/html/storage_apis/storageclass-storage-k8s-io-v1)

- [Container Storage

Interface](https://docs.redhat.com/en/documentation/openshift_container_platform/4.20/html/storage/using-container-storage-interface-csi)

```
[discrete#storage:creating-managing-storage-class::_0000000000]
```

```
OpenShift 部署 (SC) 部署 (fast-ssd, "cheap-hdd") 部署
```

PersistentVolumeClaim (PVC)
 PersistentVolume (PV)

```
[discrete#storage:creating-managing-storage-class::_000000000000000000000000]
```

=====

```
* *[*]:* [*] ([: AWS [*] `ebs.csi.aws.com`, NFS [*] `nfs.csi.k8s.io`).
```

```
* * * * *: * * * * * ([: `type: gp3, encrypted: "true"`).
```

* * :* PVC

* * : PVC

* * : * PVC

```
[discrete#storage:creating-managing-storage-class::_0000000000000000]
```

=====

A.

* PVC Pod

```
* 注意: 本示例中 (即: `AWS us-east-1a, 1b, 1c`) 部署的容器实例 A 部署在可用区Pod 部署 B
部署在可用区Pod 部署 C
```

* □□□□□□：□□□□□□□□□□□□□□□□□□□；□□□□□□□□□□□□

B. WaitForFirstConsumer (??)

* PVC Pod

* Pod: Pod PVC Pod (Pod B) Pod B

```
* 000000: 00000000 (LVM) HostPath0000000000000000
```

```
[discrete#storage:creating-managing-storage-class::_00000000]
```

=====

PVC ████████████████████████████████████

A. Delete (删除)

```
* 例: PVC を作成するPV を作成する (例: AWS EBS を作成) を作成する
* 例: 作成するCI/CD を作成する
```

B. Retain

```
* 例: PVC を作成するPV を作成する (例: AWS EBS を作成) を作成する
* 例: 作成するCI/CD を作成する
* 例: 作成するCI/CD を作成する
```

```
[discrete#storage:creating-managing-storage-class::_]
====
```

```
storageClassName を作成する PVC を作成する
```

```
例:
[source,bash,role=execute]
```

```
oc patch storageclass <your-class-name> -p '{"metadata": {"annotations": {"storageclass.kubernetes.io/is-default-class": "true"}}}'
```

```
例:
[source,bash,role=execute]
```

```
oc patch storageclass <old-default-name> -p '{"metadata": {"annotations": {"storageclass.kubernetes.io/is-default-class": "false"}}}'
```

```
例: 例 SC 1 を作成する
例[例](https://docs.redhat.com/ja/documentation/openshift_container_platform/4.20/html/storage/getting-started/default-storage-class.html) を作成する
```

```
[discrete#storage:creating-managing-storage-class::_]
====
```

A. AWS EBS (例)

```
例CSI を作成する gp3 を作成する
```

```
[source,yaml]
```

```
apiVersion: storage.k8s.io/v1 kind: StorageClass metadata: name: gp3-fast provisioner:
ebs.csi.aws.com volumeBindingMode: WaitForFirstConsumer allowVolumeExpansion: true
reclaimPolicy: Delete parameters: type: gp3 iops: "3000" throughput: "125" encrypted: "true"
```

```
OpenShift 例:
https://docs.redhat.com/ja/documentation/openshift_container_platform/4.20/html/storage/using-container-storage-interface-csi#persistent-storage-csi-ebs
```

B. 例 / LVM (例/例)

Local Storage Operator LVM Storage Operator

[source,yaml]

```
apiVersion: storage.k8s.io/v1 kind: StorageClass metadata: name: local-lvm-storage provisioner:
topolvm.io #    volumeBindingMode: WaitForFirstConsumer #
reclaimPolicy: Retain parameters: "csi.storage.k8s.io/fstype": "xfs"
```

: LVMS

C. NFS (/ RWX)

NFS CSI Driver Operator NFS

[source,yaml]

```
apiVersion: storage.k8s.io/v1 kind: StorageClass metadata: name: nfs-shared provisioner:
nfs.csi.k8s.io parameters: server: nfs-server.example.com share: /exports/data reclaimPolicy: Delete
volumeBindingMode: Immediate mountOptions: - nfsvers=4.1
```

OpenShift :

https://docs.redhat.com/ja/documentation/openshift_container_platform/4.20/html/storage/persistent-storage-using-local-storage

[discrete#storage:creating-managing-storage-class::_]

====

:

[source,bash,role=execute]

oc get sc # ()

:

[source,bash,role=execute]

oc describe sc <name>

: PVC:

[source,yaml]

```
kind: PersistentVolumeClaim apiVersion: v1 metadata: name: test-pvc spec: accessModes: [
```

"ReadWriteOnce"] resources: { requests: { storage: 1Gi } } storageClassName: gp3-fast

OpenShift 参考:

https://docs.redhat.com/ja/documentation/openshift_container_platform/4.20/html/storage/configuring-persistent-storage

[discrete#storage:creating-managing-storage-class::_参考]

==== 参考

* 例 1: PVC 参考

** 例 A: volumeBindingMode = WaitForFirstConsumer 参考 Pod = PVC 参考

*** 例: 例 PVC 参考 Pod 参考

** 例 B: Pod 参考

*** 例: 'oc get pods' 参考Pod 参考

* 例 2: Pod 参考 (参考)

** 例: 参考

** 例: 参考 (例: 'AWS us-east-1a, 1b, 1c') 例volumeBindingMode: Immediate 参考

A 参考Pod 参考 B 参考

** 例: PVC 参考WaitForFirstConsumer 参考

* 例 3: "参考"

** 例: provisioner 参考 CSI 参考

** 例: 参考 (例: openshift-cluster-csi-drivers) 参考

* 例 4: 参考

** 例: StorageClass = allowVolumeExpansion = false 参考

** 例: StorageClass = true 参考

NOTE: 参考

[discrete#storage:creating-managing-storage-class::_参考]

==== 参考

参考:

* OpenShift 参考

* 参考

* 'Immediate' = 'WaitForFirstConsumer' 参考

* AWS EBS参考 LVM参考NFS 参考

* PVC 参考

[discrete#storage:creating-managing-storage-class::_参考]

==== 参考

* [OpenShift -

参考](https://docs.redhat.com/ja/documentation/openshift_container_platform/4.20/html/storage/configuring-persistent-storage)

* [OpenShift - Container Storage

Interface](https://docs.redhat.com/ja/documentation/openshift_container_platform/4.20/html/storage/using-container-storage-interface-csi)

```

:!navtitle:

:docname: multi-disk-configuration
:page-module: storage
:page-relative-src-path: multi-disk-configuration.adoc
:page-origin-url: https://github.com/RedHatQuickCourses/ocp-virt-cookbook_ja
:page-origin-start-path:
:page-origin-refname: main
:page-origin-reftype: branch
:page-origin-refhash: (worktree)
:navtitle: 仮想マシン VM
[#storage:multi-disk-configuration]
=== 仮想マシン VM

[discrete#storage:multi-disk-configuration::_00]
==== 00

仮想マシンを構築する際には、OS のインストールと、OS 上で仮想マシン VM を構築する。

2つのステップ: 1. OS のインストール (2. 仮想マシン VM の構築)

[discrete#storage:multi-disk-configuration::_0000]
==== 0000

* **OpenShift 4.18+** は OpenShift Virtualization をサポートする。
* **root** (`system:admin` ユーザー)
* **`oc` CLI** は **`virtctl` CLI** を使用する。
* **root** (例: `lvms-vg1`) を使用する。

インストール:

```

OpenShift 4.20から4.21

```

[discrete#storage:multi-disk-configuration::_0000_1_仮想マシン VM]
==== 0000 1: 仮想マシン VM

KubeVirt VM を構築する:

* **`spec.template.spec.domain.devices.disks`** は OS のインストール (インストール) を行う。
* **`spec.template.spec.volumes`** は 仮想マシン VM
(PVC、DataVolume、containerDisk、cloudInitNoCloud) を使用する。
* 仮想マシン `name` は `name` を使用する。

インストール `"/dev/vda"` `"/dev/vdb"` を使用する。

[discrete#storage:multi-disk-configuration::_0000_2_仮想マシン VM]
==== 0000 2: 仮想マシン VM

```



```
[source,bash,role=execute]
```

```
oc apply -f - <<EOF apiVersion: kubevirt.io/v1 kind: VirtualMachine metadata: name: multi-disk-vm
namespace: multi-disk-demo spec: runStrategy: Always template: metadata: labels:
kubevirt.io/domain: multi-disk-vm spec: domain: devices: disks: - name: rootdisk disk: bus: virtio
bootOrder: 1 - name: datadisk disk: bus: virtio - name: cloudinitdisk disk: bus: virtio interfaces: -
name: default masquerade: {} resources: requests: memory: 1Gi networks: - name: default pod: {}
volumes: - name: rootdisk dataVolume: name: multi-disk-rootdisk - name: datadisk dataVolume:
name: multi-disk-datadisk - name: cloudinitdisk cloudInitNoCloud: userData: | #cloud-config
password: redhat chpasswd: expire: false dataVolumeTemplates: - metadata: name: multi-disk-
rootdisk spec: pvc: accessModes: - ReadWriteOnce resources: requests: storage: 30Gi
storageClassName: lvms-vg1 source: registry: pullMethod: node url:
docker://quay.io/containerdisks/fedora:latest - metadata: name: multi-disk-datadisk spec: pvc:
accessModes: - ReadWriteOnce resources: requests: storage: 10Gi storageClassName: lvms-vg1-
retain source: blank: {} EOF
```

```
[discrete#storage:multi-disk-configuration::_00000000]
===== 00000000

* **`bootOrder: 1`** 00rootdisk  VM  OS 000000000000000000000000
* **`source.registry`** 00rootdisk  Fedora 000000000000000000000000
* **`source.blank: {}`** 00datadisk 000000000000000000000000
* `dataVolumeTemplates` 000000000000 `storageClassName` 00000000
* `devices.disks` 00000000 `volumes` 000000000000000000000000 `dataVolumeTemplates` 000000000000000000000000

[discrete#storage:multi-disk-configuration::_0000_5_vm_00000000]
===== 0000 5: VM 0000000000

000 DataVolume 00000VM 000000000000:

[source,bash,role=execute]
```

```
oc get dv -n multi-disk-demo -w
```

```
000 `Succeeded` 000000VM 000000:

[source,bash,role=execute]
```

```
oc get vm -n multi-disk-demo
```

```
00000000:
```

```
NAME AGE STATUS READY multi-disk-vm 3m Running True
```


lsblk

lsblk:

```
NAME    MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
vda      252:0    0   30G  0 disk
|-vda1   252:1    0    1M  0 part
|-vda2   252:2    0   1.9G  0 part [SWAP]
|-vda3   252:3    0    1G  0 part /boot
`-vda4   252:4    0  27.1G  0 part /
vdb      252:16    0   10G  0 disk
vdc      252:32    0    1M  0 disk
```

lsblk vdb (10G disk) filesystem XFS filesystem:

```
sudo mkfs.xfs /dev/vdb
```

lsblk vdb (10G disk) filesystem XFS filesystem:

```
sudo mkdir -p /mnt/data
sudo mount /dev/vdb /mnt/data
```

lsblk:

```
df -h /mnt/data
```

lsblk:

```
Filesystem      Size  Used Avail Use% Mounted on
/dev/vdb         10G   105M   9.9G   2% /mnt/data
```

lsblk vdb (10G disk) filesystem XFS filesystem:

```
sudo sh -c "echo 'Multi-disk test data' > /mnt/data/test.txt"
```

lsblk:

```
cat /mnt/data/test.txt
```

lsblk vdb (10G disk) filesystem XFS filesystem: /etc/fstab filesystem:

```
sudo sh -c "echo '/dev/vdb /mnt/data xfs defaults 0 0' >> /etc/fstab"
```

Ctrl+]

7:

PVC PersistentVolume

```
oc get pvc multi-disk-rootdisk -n multi-disk-demo -o jsonpath='{StorageClass:
{.spec.storageClassName}, Volume: {.spec.volumeName}, Capacity:
{.status.capacity.storage}]'
```

```
oc get pvc multi-disk-datadisk -n multi-disk-demo -o jsonpath='{StorageClass:
{.spec.storageClassName}, Volume: {.spec.volumeName}, Capacity:
{.status.capacity.storage}]'
```

(:)

VM

	SSD (OS +)	HDD ()
	(OS)	SSD ()
VM	(OS)	()
/	(OS)	()

VM**PVC** **DataVolumes**

```
oc delete namespace multi-disk-demo
```

2:

```
oc delete storageclass lvms-vg1-retain
```

lvms-vg1-retain **reclaimPolicy: Retain** **PVC** **PersistentVolume** **PV**

```
oc get pv | grep Released
```

削除する PV を指定する必要があります:

```
oc delete pv <pv-name>
```

例

削除する PV:

- 削除する PV の名前を指定して VM を削除
- `dataVolumeTemplates` を指定して削除する PV を指定
- 削除する PV の名前を指定して VM を削除
- 削除する PV の名前を指定して削除
- OS を指定して削除
- VM を指定して削除

例

- [KubeVirt](#) - KubeVirt を削除
- [OpenShift](#) - OpenShift を削除
- [LVM Storage Operator](#)

OpenShift Virtualization

UDN

OpenShift Virtualization (UDN) localnet 2
VLAN Linux
OpenShift Virtualization (UDN) localnet 2
VLAN Linux

localnet

OpenShift Virtualization (UDN) localnet 2

- cloud-init DHCP IP
- cloud-init IP
- (UDN) 2
- localnet
- VLAN
- 2
- Linux
- PCI SR-IOV
- OVS
-

OpenShift Virtualization

OpenShift Virtualization (UDN) localnet 2

- (UDN): 2
- localnet:
- 2:
- Linux
- SR-IOV: PCI
- OVN-Kubernetes

localnet

OpenShift Virtualization (UDN) localnet 2

- OpenShift 4.19+ OpenShift Virtualization
- NMState (OpenShift Virtualization)

- [OpenShift Container Platform 3.11.0 安装指南](#)
- [OpenShift Container Platform 3.11.0 安装指南](#)
- [OpenShift Container Platform 3.11.0 安装指南](#)

OpenShift

OpenShift Container Platform 3.11.0 安装指南

- [OpenShift Container Platform 3.11.0 安装指南](#)
- [OpenShift Container Platform 3.11.0 安装指南](#)

OpenShift

- [VM 安装 DNS - OpenShift Container Platform 3.11.0 安装指南](#)
- [OpenShift Virtualization 3.11.0 安装指南](#)

IP地址.adoc

OpenShift Container Platform 3.11.0 安装指南.adoc

UDN安装指南.adoc

OpenShift Container Platform 3.11.0 安装指南.adoc

OpenShift Container Platform 3.11.0 安装指南.adoc

CUDN安装指南.adoc

OpenShift 2.11.0 安装指南.adoc

Linux 安装指南.adoc

SIRIUS 安装指南.adoc

VM 安装指南.adoc

OVS 安装指南.adoc

○○○○○○○○

○○

○○○○○○○○OpenShift Virtualization ○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○DNS
○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○ VM ○○○○○○○○○○○○○

○○○○

OpenShift Virtualization ○○○○○○○○○○○○○○○○○○○○○○○○○

- VM-to-VM □ Pod-to-VM ○○○○○ DNS ○○
- Linux □ Windows VM ○○○○○○○○○○○○○
- ○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○
- ○○○○○○○○○○○○○○○○○○○○○○○○○○○○○
- ○○○ VM ○○○○○○○○○○○○○○○○○○○○○○○○○○○○○
- ○○○○○○○○○○○○○○○○○ VM ○○○○○○○○

○○○○

VM ○○○○○○○○○○○○○○○○○○○○○○○

- OpenShift 4.18+ □ OpenShift Virtualization ○○○○○○○○○○○○○○○
- ○○○○○○○○○○○○○○○○○○○
- YAML ○○○○
- CLI ○○○○○○○○○○○○○: **oc**, **virtctl**, **kubectl**

○○○○

○○○○○○○ **DNS** ○○

Kubernetes ○○○○○○○○○○○○○ VM-to-VM □ Pod-to-VM ○○○○○ DNS ○○○○○○○○○○○○○○○

○○○○○○○○○○○ **Linux VM** ○○○○○○

○○○○VNC□virtctl ○○○ SSH□NodePort ○○○○○○○○○○○○○○○○○○○○○○○○○○○○○Linux VM
○○○○○○○○○○○○○○○○

○○○○○○○○○ **Linux VM** ○○○○○○○○○○○○

VM ○○○

Windows 11 VM □ Red Hat OpenShift □ RDP ○○○○○○○○○○○○

○○ Remote Desktop Protocol (RDP) ○○○ Windows VM ○○○○○ LoadBalancer ○ NodePort
○○○○

OpenShift

HyperConverged OpenShift Virtualization
 (virt-controller virt-api virt-handler CDI)

VM

Pod VM

virtctl VM

OpenShift Virtualization VM

-
- VM
-
- CPU NUMA hugepages VM
- VM -

- OpenShift Virtualization
- init
- KubeVirt

DNS.adoc

Linux.adoc

Linux.adoc

Windows.adoc

WindowsVirtio.adoc

.adoc

.adoc

□□□□□□□□□□□□□□□□□□□□□□.adoc

□□□□□□□□□□□□.adoc

VM 管理

概要

OpenShift Virtualization は、Kubernetes 上で VM を実行するためのプラットフォームです。CPU、メモリ、NUMA などのリソースを管理します。

特徴

- CPU の QoS を提供
- VM vCPUs の CPU 使用率を制限
- VM の NUMA ノードを指定
- 大規模なメモリ使用をサポート

インストール

VM をインストールするには、以下の手順に従ってください。

- OpenShift 4.18+ に OpenShift Virtualization をインストール
- Kubernetes の設定を確認
- CLI ツール: `oc`, `virtctl`, `kubectrl`
- CPU の hugepages を設定

管理

QoS

CPU の QoS を管理するには、LimitRanges を使用します。

CPU

Kubernetes の CPU リソースを vCPUs の単位で管理します。CPU の使用率を制限し、CPU の性能を確保します。

NUMA

NUMA ノードを指定し、TLB を有効にします。NUMA の性能を最大化します。

メモリ

CPU、NUMA、hugepages、RT、SCHED_FIFO の設定を確認します。

トラブルシューティング

VM のエラーメッセージを確認してください。

1. **QoS** — CPU QoS
2. **CPU** — CPU vCPUs
3. **NUMA** — NUMA
4. — RT SCHED_FIFO

- -
- -

- OpenShift Virtualization
- KubeVirt

QoS.adoc

CPU.adoc

NUMA.adoc

.adoc

インストール

qcow2 を使用して OpenShift Virtualization をインストールする 4 ステップ

コマンド	説明	関連リソース
virtctl image-upload	イメージをアップロードする	イメージを DataVolume/PVC にアップロードする
DataVolume with HTTP/S3	イメージを HTTP/S3 から取得する	Web からイメージを DataVolume にアップロードする
ContainerDisk	イメージを ContainerDisk から取得する	qcow2 イメージを ContainerDisk にアップロードする
VM with DataVolumeTemplate	イメージを VM にマウントする	VM に DataVolumeTemplate をマウントする

このセクションでは、**virtctl image-upload** を使用してイメージをアップロードする方法について説明します。

ステップ 1: qcow2 をダウンロード

qcow2 を使用して OpenShift Virtualization をインストールする

イメージをダウンロードする

イメージをダウンロードして qcow2 フォーマットに変換する

```
# インストール (例)
curl -LO
https://download.fedoraproject.org/pub/fedora/linux/releases/43/Cloud/x86_64/images/Fedora-Cloud-Base-Generic-43-1.6.x86_64.qcow2

# 確認
ls -lh Fedora-Cloud-Base-Generic-43-1.6.x86_64.qcow2
```

イメージを確認する (オプション)

qemu-img を使用してイメージを確認する

```
qemu-img info Fedora-Cloud-Base-Generic-43-1.6.x86_64.qcow2
```

出力:

```
image: Fedora-Cloud-Base-Generic-43-1.6.x86_64.qcow2
file format: qcow2
size: 5 GiB (5368709120 bytes)
virtual size: 420 MiB
```

DataVolume を作成する

ステップ 2: 名前空間を作成

VM を作成する前に

```
oc create namespace imported-vms
```

ステップ 3: virtctl を使って qcow2 をアップロード

virtctl を使って qcow2 をアップロードして PVC を作成する

```
virtctl image-upload pvc fedora-imported-disk \
  --image-path=./Fedora-Cloud-Base-Generic-43-1.6.x86_64.qcow2 \
  --size=10Gi \
  --namespace=imported-vms \
  --insecure \
  --force-bind
```

オプション:

- **pvc fedora-imported-disk** - 作成する PVC の名前
- **--image-path** - qcow2 ファイルのパス
- **--size** - PVC のサイズ (qcow2 ファイルのサイズ)
- **--namespace** - 名前空間
- **--insecure** - TLS を無視する (デフォルトは true)
- **--force-bind** - StorageClasses が **WaitForFirstConsumer** の場合、PVC が作成された後 (AWS EBS/Azure Disk など) 自動的にディスクをバインドする

例: **volumeBindingMode: WaitForFirstConsumer** の場合、StorageClasses が **--force-bind** を指定しない限り、PVC が作成された後自動的にディスクをバインドする

出力:

```
PVC imported-vms/fedora-imported-disk をアップロード
https://cdi-uploadproxy-openshift-cnv.apps... をアップロード

420.00 MiB / 420.00 MiB [=====] 100.00% 1m30s

アップロードが完了しました
アップロードが完了しました
```

アップロードされたディスクは 1GB のディスクとして 1-3 分以内に

PVC

□□□□□□□□PVC □ "Pending" □□ "Bound" □□□□□□:

`cdi-upload` `XXXXXXXXXXXXXX`:

Figure 4: Performance of the VM

□□□□□□□□□□□□PVC □□□□□□□□□□□□ VM □□□□□□

```
00: 0000 cloud-init 00000000000000000000000000000000 SSH 000000000000
```

vm-from-imported-qcow2.yaml

135


```

        bus: virtio
        name: rootdisk
      - disk:
          bus: virtio
          name: cloudinitdisk
    interfaces:
      - masquerade: {}
        name: default
    rng: {}
    machine:
      type: pc-q35-rhel9.4.0
    memory:
      guest: 4Gi
    networks:
      - name: default
        pod: {}
    volumes:
      - name: rootdisk
        persistentVolumeClaim:
          claimName: fedora-imported-disk
      - name: cloudinitdisk
        cloudInitNoCloud:
          userData: |-
            #cloud-config
            user: fedora
            password: fedora123
            chpasswd: { expire: false }
            ssh_pwauth: true

```

VM 创建成功:

```
oc apply -f vm-from-imported-qcow2.yaml
```

VM 启动:

```
virtctl start fedora-imported-vm -n imported-vms
```

步骤 5: VM 创建成功

VM 创建成功:

```
oc get vmi -n imported-vms --watch
```

查看 VM 状态:

NAME	AGE	PHASE	IP	NODENAME	READY
------	-----	-------	----	----------	-------

fedora-imported-vm	15s	Scheduling			False
fedora-imported-vm	30s	Scheduled		worker-01	False
fedora-imported-vm	45s	Running	10.128.2.54	worker-01	True

VM 確認:

```
virtctl console fedora-imported-vm -n imported-vms
```

cloud-init 確認 (fedora) 確認 (fedora123) 確認

確認:

```
# 確認
cat /etc/os-release

# 確認
lsblk
```

確認 **Ctrl+]** 確認

確認

virtctl image-upload 確認

2: HTTP 確認 DataVolume

qcow2 確認 HTTP/HTTPS 確認 CDI 確認 DataVolume
確認 Web 確認

確認:

datavolume-http-import.yaml 確認

```
apiVersion: cdi.kubevirt.io/v1beta1
kind: DataVolume
metadata:
  name: fedora-http-imported
  namespace: imported-vms
  annotations:
    cdi.kubevirt.io/storage.bind.immediate.requested: "true"
spec:
  source:
    http:
      url:
        "https://download.fedoraproject.org/pub/fedora/linux/releases/43/Cloud/x86_64/images/Fedora-Cloud-Base-Generic-43-1.6.x86_64.qcow2"
  storage:
    resources:
```

```
requests:
  storage: 10Gi
```

```
0: PVC cdi.kubevirt.io/storage.bind.immediate.requested:
"true" WaitForFirstConsumer StorageClass
```

DataVolume CDI

```
oc apply -f datavolume-http-import.yaml
oc get dv -n imported-vms --watch
```

NAME	PHASE	PROGRESS	RESTARTS	AGE
fedora-http-imported	ImportScheduled	N/A		5s
fedora-http-imported	ImportInProgress	12.5%		15s
fedora-http-imported	ImportInProgress	67.3%		45s
fedora-http-imported	Succeeded	100.0%		1m30s

DataVolume Succeeded XXXXXXXXXXXX VM XXXXXXXX:

```
volumes:
  - name: rootdisk
    dataVolume:
      name: fedora-http-imported
```

注意: 本表為 CI/CD 系統提供 S3 URL 資訊

□□□□ 3: ContainerDisk

qcow2

Containerfile □□□□□□:

```
FROM scratch
ADD Fedora-Cloud-Base-Generic-43-1.6.x86_64.qcow2 /disk/
```

□ □ □ □ □ □ □ □ □ □ □ :

```
podman build -t quay.io/youruser/fedora-disk:v1 .
podman push quay.io/youruser/fedora-disk:v1
```

VM 00000000:

```
volumes:
  - name: rootdisk
    containerDisk:
      image: quay.io/youruser/fedora-disk:v1
```

[ContainerDisk VM]

□□□□ 4: DataVolumeTemplate □□□□□□□□□□□□ VM

```

DataVolumeTemplate   VM   DataVolume   VM
VM

```

000000000000: - VM 000000000000 1 0000000000000000 - GitOps 0000000000000000 - DataVolume
0000000000 VM 000000000000

vm-with-upload-datavolume.yaml □□□□□□

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: vm-with-upload
  namespace: imported-vms
spec:
  dataVolumeTemplates:
    - apiVersion: cdi.kubevirt.io/v1beta1
      kind: DataVolume
      metadata:
        name: vm-with-upload-disk
      spec:
        source:
          upload: {}
        storage:
          resources:
            requests:
              storage: 10Gi
  runStrategy: RerunOnFailure
  template:
    metadata:
      labels:
        kubevirt.io/domain: vm-with-upload
    spec:
      domain:
        cpu:
          cores: 2
        devices:
          disks:
            - disk:
                bus: virtio
                name: rootdisk
```

```

    interfaces:
      - masquerade: {}
        name: default
    rng: {}
  machine:
    type: pc-q35-rhel9.4.0
  memory:
    guest: 4Gi
  networks:
    - name: default
      pod: {}
  volumes:
    - dataVolume:
        name: vm-with-upload-disk
      name: rootdisk

```

VM `oc apply -f vm-with-upload-datavolume.yaml`:

```
oc apply -f vm-with-upload-datavolume.yaml
```

~~~~~

```

virtctl image-upload dv vm-with-upload-disk \
  --image-path=./Fedora-Cloud-Base-Generic-43-1.6.x86_64.qcow2 \
  --size=10Gi \
  --namespace=imported-vms \
  --insecure \
  --force-bind

```

~~~~~

```
virtctl start vm-with-upload -n imported-vms
```

~~~~~

**Summary:** - 1 `oc apply` VM `oc apply` - `DataVolume` `oc apply` VM `oc apply` - `Infrastructure-as-Code` `oc apply`

**Notes:** - `PVC` `oc apply` - `oc apply` VM `oc apply` - `DataVolume/virtctl` `oc apply`

~~~~~

~~~~~

- `oc apply` - `oc apply`
- `oc apply` - `virt-sparsify` `qcow2` `oc apply`
- `oc apply` - `oc apply`

- **cloud-init** `cloud-init` - `cloud-init` `cloud-init`

cloud-init

- `cloud-init` - VM `cloud-init` `cloud-init`
- `cloud-init` - `cloud-init` `cloud-init`
- `cloud-init` - PVC `cloud-init`

cloud-init

- `cloud-init` - `cloud-init` `qcow2` `cloud-init`
- **TLS** `cloud-init` - `cloud-init` `--insecure` `cloud-init`
- `cloud-init` - RBAC `cloud-init`
- `cloud-init` - `cloud-init` `cloud-init`

cloud-init

- `cloud-init` - `cloud-init` `cloud-init`
- `cloud-init` - `cloud-init` `cloud-init`
- `cloud-init` - `cloud-init` `cloud-init`

cloud-init

cloud-init **"Waiting for PVC upload pod to be ready"** `cloud-init`

PVC `"WaitForFirstConsumer"` `cloud-init`

PVC `cloud-init`:

```
oc get pvc -n imported-vms
oc describe pvc fedora-imported-disk -n imported-vms
```

cloud-init `"waiting for first consumer to be created before binding"` `cloud-init`  
**volumeBindingMode: WaitForFirstConsumer** `cloud-init`

cloud-init: `virtctl` `cloud-init` `--force-bind` `cloud-init`:

```
virtctl image-upload pvc fedora-imported-disk \
  --image-path=./image.qcow2 \
  --size=10Gi \
  --namespace=imported-vms \
  --insecure \
  --force-bind
```

cloud-init PVC `cloud-init`:

```
oc delete pvc fedora-imported-disk -n imported-vms
```

❗ **--force-bind** を指定してください

**"Unable to get PVC Prime name from PVC"** の対処法

CDI の `DataVolume` を `PVC` にマウントする際、`PVC` の名前が正しく指定されていない場合があります。

```
# PVC をマウント (DataVolume をマウント)
virtctl image-upload pvc <pvc-name> \
  --image-path=./image.qcow2 \
  --size=10Gi \
  --namespace=imported-vms \
  --insecure
```

確認: \* **CDI** のデプロイメント: `cdi-deployment` と `cdi-operator` が正常に実行されているかを確認してください。

CDI のログを確認:

```
oc get pods -n openshift-cnv | grep cdi
```

確認: CDI の Pod が正常に実行されているかを確認してください。

CDI のログを確認:

```
oc get pods -n openshift-cnv | grep cdi-uploadproxy
oc logs -n openshift-cnv <cdi-uploadproxy-pod>
```

確認: CDI の Pod が正常に実行されているかを確認してください。

**PVC** のサイズを確認

**--size** を指定して `qcow2` のイメージを作成:

```
qemu-img info image.qcow2 | grep "virtual size"
```

CDI の `PVC` のサイズを確認: **--size** を指定してください。

```
oc delete pvc fedora-imported-disk -n imported-vms
```

```
# 再度 --size を指定
virtctl image-upload pvc fedora-imported-disk \
```

```
--image-path=./image.qcow2 \  
--size=20Gi \  
--namespace=imported-vms \  
--insecure
```

VM

VM:

```
oc describe vm fedora-imported-vm -n imported-vms  
oc get events -n imported-vms --field-selector involvedObject.name=fedora-imported-vm
```

:

```
virtctl console fedora-imported-vm -n imported-vms
```

: \* - \* - VM **machine.type** \* -

:

```
# VM  
oc delete vm fedora-imported-vm -n imported-vms  
  
# PVC  
oc delete pvc fedora-imported-disk -n imported-vms  
  
#  
oc delete namespace imported-vms
```

: PVC

:

- OpenShift Virtualization **qcow2**
- **virtctl image-upload pvc** **qcow2**
- VM
- 
- HTTPContainerDiskDataVolumeTemplate
-



- [illegible]

qcow2 KVM OpenShift Virtualization

□ □ □ □ □ □

- **IP 地址池** - 虚拟机 IP 地址池
- **IP 地址** - 虚拟机 IP 地址
- **IP 地址池** - 虚拟机 IP 地址池

11

- Containerized Data Importer (CDI)
- virtctl □□□□□□

|  |  |  |  |  |  |  |  |  |
|--|--|--|--|--|--|--|--|--|
|  |  |  |  |  |  |  |  |  |
|--|--|--|--|--|--|--|--|--|

11

OpenShift Virtualization (VM)

□ □ □ □ □ □ □ □ □ □ □ □

[illegible][illegible]

□ □ □ □

- OpenShift 4.18+ → OpenShift Virtualization
- CLI ツール: `oc`, `virtctl`, `kubectl`
- 仮想マシンを管理 (例: `lvms-vg1`)
- Linux 環境
- cloud-init

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □

000000000000000000000000 . 00 OS 0000000000 VM 00000000 . VM 0000000000 (00000000000000000000) . VM  
 00000000 (000000000000) . VM 00000000000000000000 . 00000000000000000000 . 00000000000000 VM 00000000

## 例 1: VM 作成

以下のコマンドを実行:

```
oc create namespace golden-images
```

以下のコマンドで Fedora VM を作成:

[例] cloud-init を利用して SSH 接続できるようにする

**golden-image-base-vm.yaml** の内容

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: base-fedora-vm
  namespace: golden-images
spec:
  dataVolumeTemplates:
    - apiVersion: cdi.kubevirt.io/v1beta1
      kind: DataVolume
      metadata:
        name: base-fedora-vm-disk
      spec:
        sourceRef:
          kind: DataSource
          name: fedora
          namespace: openshift-virtualization-os-images
        storage:
          resources:
            requests:
              storage: 30Gi
  runStrategy: RerunOnFailure
  template:
    metadata:
      labels:
        kubevirt.io/domain: base-fedora-vm
    spec:
      domain:
        cpu:
          cores: 2
        devices:
          disks:
            - disk:
                bus: virtio
                name: rootdisk
            - disk:
                bus: virtio
                name: cloudinitdisk
```

```

    interfaces:
      - masquerade: {}
        name: default
    rng: {}
  machine:
    type: pc-q35-rhel9.4.0
  memory:
    guest: 4Gi
  networks:
    - name: default
      pod: {}
  volumes:
    - dataVolume:
        name: base-fedora-vm-disk
        name: rootdisk
    - cloudInitNoCloud:
        userData: |-
          #cloud-config
          user: fedora
          password: fedora123
          chpasswd: { expire: false }
          ssh_pwauth: true
        name: cloudinitdisk

```

`base-fedora-vm.yaml` ファイルを作成:

```
oc apply -f base-fedora-vm.yaml
```

VM の作成を確認:

```
oc get dv -n golden-images
```

確認:

| NAME                | PHASE           | PROGRESS | RESTARTS | AGE |
|---------------------|-----------------|----------|----------|-----|
| base-fedora-vm-disk | CloneInProgress | 45.2%    |          | 2m  |

確認 **Succeeded** VM の作成:

```
oc get vmi -n golden-images
```

確認:

| NAME | AGE | PHASE | IP | NODENAME | READY |
|------|-----|-------|----|----------|-------|
|------|-----|-------|----|----------|-------|

base-fedora-vm 4m Running 10.128.1.151 ocplab01 True

[00] 3-5

## 2: VM

VM:

```
virtctl console base-fedora-vm -n golden-images
```

fedora fedora123

:

```
#
sudo dnf update -y

# Nginx Web
sudo dnf install -y nginx vim htop curl wget git

# Nginx
sudo systemctl enable nginx

#
sudo bash -c 'cat > /usr/share/nginx/html/index.html << EOF
<!DOCTYPE html>
<html>
<head>
  <title> VM</title>
</head>
<body>
  <h1> VM</h1>
  <p> VM</p>
  <p>Nginx</p>
</body>
</html>
EOF'

#
sudo firewall-cmd --permanent --add-service=http
sudo firewall-cmd --reload
```

## 3: VM

:

```
#
```

```

sudo dnf clean all

# 清除日志
sudo find /var/log -type f -exec truncate -s 0 {} \;

# 清除 ID
sudo rm -f /etc/machine-id
sudo touch /etc/machine-id

# cloud-init 清除
sudo cloud-init clean --logs --seed

# 清除历史
history -c
cat /dev/null > ~/.bash_history

# 同步并退出
sync
exit

```

按 **Ctrl+J** 继续

## 步骤 4: VM 停止

在 OpenShift 控制台中执行以下命令以停止 VM：

```
virtctl stop base-fedora-vm -n golden-images
```

在 OpenShift 控制台中执行以下命令以查看 VM 状态：

```
oc get vm -n golden-images
```

输出：

| NAME           | AGE | STATUS  | READY |
|----------------|-----|---------|-------|
| base-fedora-vm | 15m | Stopped | False |

## 步骤 5: 创建 VM 镜像数据源

在 OpenShift 控制台中执行以下命令以创建 VM 镜像数据源：

在 OpenShift 控制台中执行以下命令：

**golden-image-datasource.yaml** 文件内容

```
apiVersion: cdi.kubevirt.io/v1beta1
```

```
kind: DataSource
metadata:
  name: fedora-nginx-golden
  namespace: golden-images
spec:
  source:
    pvc:
      name: base-fedora-vm-disk
      namespace: golden-images
```

**fedora-golden-datasource.yaml** を作成:

```
oc apply -f fedora-golden-datasource.yaml
```

確認:

```
oc get datasource -n golden-images
```

結果:

| NAME                | AGE |
|---------------------|-----|
| fedora-nginx-golden | 5s  |

**ステップ 6: VM を作成**

作成した **DataSource** を利用して **VM** を作成します。

以下の **VM** を作成:

**webserver-vm-from-golden.yaml** を作成

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: webserver-vm-1
  namespace: golden-images
spec:
  dataVolumeTemplates:
    - apiVersion: cdi.kubevirt.io/v1beta1
      kind: DataVolume
      metadata:
        name: webserver-vm-1-disk
      spec:
        sourceRef:
          kind: DataSource
          name: fedora-nginx-golden
```

```

    namespace: golden-images
  storage:
    resources:
      requests:
        storage: 30Gi
  runStrategy: RerunOnFailure
  template:
    metadata:
      labels:
        kubevirt.io/domain: webserver-vm-1
    spec:
      domain:
        cpu:
          cores: 2
        devices:
          disks:
            - disk:
                bus: virtio
                name: rootdisk
            - disk:
                bus: virtio
                name: cloudinitdisk
          interfaces:
            - masquerade: {}
              name: default
          rng: {}
        machine:
          type: pc-q35-rhel9.4.0
        memory:
          guest: 4Gi
      networks:
        - name: default
          pod: {}
      volumes:
        - dataVolume:
            name: webserver-vm-1-disk
            name: rootdisk
        - cloudInitNoCloud:
            userData: |-
              #cloud-config
              hostname: webserver-vm-1
              fqdn: webserver-vm-1.example.com
            name: cloudinitdisk

```

2 VM 创建:

**webserver-vm-2.yaml** 创建

```

apiVersion: kubevirt.io/v1
kind: VirtualMachine

```

```

metadata:
  name: webserver-vm-2
  namespace: golden-images
spec:
  dataVolumeTemplates:
    - apiVersion: cdi.kubevirt.io/v1beta1
      kind: DataVolume
      metadata:
        name: webserver-vm-2-disk
      spec:
        sourceRef:
          kind: DataSource
          name: fedora-nginx-golden
          namespace: golden-images
        storage:
          resources:
            requests:
              storage: 30Gi
  runStrategy: RerunOnFailure
  template:
    metadata:
      labels:
        kubevirt.io/domain: webserver-vm-2
    spec:
      domain:
        cpu:
          cores: 2
        devices:
          disks:
            - disk:
                bus: virtio
                name: rootdisk
            - disk:
                bus: virtio
                name: cloudinitdisk
          interfaces:
            - masquerade: {}
              name: default
          rng: {}
        machine:
          type: pc-q35-rhel9.4.0
        memory:
          guest: 4Gi
      networks:
        - name: default
          pod: {}
      volumes:
        - dataVolume:
            name: webserver-vm-2-disk
            name: rootdisk
        - cloudInitNoCloud:

```



```
userData: |-
  #cloud-config
  hostname: webserver-vm-2
  fqdn: webserver-vm-2.example.com
name: cloudinitdisk
```

VM 00000000:

```
oc apply -f webserver-vm-1.yaml
oc apply -f webserver-vm-2.yaml
```

□□□□□□□□□□□□:

```
oc get dv -n golden-images
```

| NAME                | PHASE           | PROGRESS | RESTARTS | AGE |
|---------------------|-----------------|----------|----------|-----|
| base-fedora-vm-disk | Succeeded       | 100.0%   |          | 15m |
| webserver-vm-1-disk | CloneInProgress | 72.5%    |          | 3m  |
| webserver-vm-2-disk | CloneInProgress | 71.8%    |          | 3m  |

1. VM 3-5

Figure 7: VM

**VM 0000000000000000:**

```
oc get vmi -n golden-images
```

| NAME           | AGE | PHASE   | IP           | NODENAME | READY |
|----------------|-----|---------|--------------|----------|-------|
| webserver-vm-1 | 5m  | Running | 10.128.1.152 | ocplab01 | True  |
| webserver-vm-2 | 5m  | Running | 10.128.1.153 | ocplab01 | True  |

# 1 VM 部署 Nginx 部署

```
virtctl console webserver-vm-1 -n golden-images
```

## Nginx 安装配置:

```
sudo systemctl status nginx
```

nginx.service (nginx) の状態を調べる

Web ブラウザで確認:

```
curl localhost
```

nginx が HTML を返すことを確認する

nginx を再起動するには **Ctrl+]** を押す

確認する

nginx のインストールと起動:

1. 仮想マシン (VM) を起動する
2. 接続を確認する
3. VM を再起動する
4. nginx をインストールする (例: **fedora-nginx-golden-v2**)
5. VM を再起動する

接続を確認する: - **fedora-nginx-golden-v1** - **fedora-nginx-golden-v2** - **fedora-nginx-golden-2025-10**

確認する

確認する

- nginx のインストールと起動
- nginx の再起動
- nginx のインストールと起動
- nginx の再起動
- nginx のインストールと起動

確認

- nginx のインストールと起動
- SSH で接続する
- nginx のインストールと起動
- cloud-init をインストールする
- nginx のインストールと起動

## OpenShift

- OpenShift Container Platform
- OpenShift Kubernetes Platform
- OpenShift Serverless

## OpenShift

- OpenShift Container Platform
- OpenShift Kubernetes Platform
- OpenShift Serverless
- OpenShift Virtualization
- cloud-init VM (SSH) (SSH) (SSH)

## OpenShift

OpenShift Container Platform (OCP) is a Kubernetes-based container management platform that provides a secure and scalable environment for running containerized applications.

## OpenShift

OpenShift Container Platform (OCP) is a Kubernetes-based container management platform that provides a secure and scalable environment for running containerized applications.

```
# VM
oc delete vm webserver-vm-1 webserver-vm-2 base-fedora-vm -n golden-images

# datasource
oc delete datasource fedora-nginx-golden -n golden-images

# namespace
oc delete namespace golden-images
```

## OpenShift

OpenShift Container Platform (OCP) is a Kubernetes-based container management platform that provides a secure and scalable environment for running containerized applications.

OpenShift Container Platform (OCP) is a Kubernetes-based container management platform that provides a secure and scalable environment for running containerized applications.

## OpenShift

- OpenShift Virtualization
- KubeVirt DataVolume
- OpenShift Serverless

## OpenShift Container Platform

OpenShift

Virtualization

Container

OS

OpenShift Container Platform is a Kubernetes-based container management platform that runs on top of Linux or Windows operating systems.

For

- [\[vm-lifecycle:golden-images-troubleshooting:::troubleshooting\]](#)
- [\[vm-lifecycle:golden-images-troubleshooting:::troubleshooting\]](#)
- [\[vm-lifecycle:golden-images-troubleshooting:::troubleshooting\]](#)

OpenShift Container Platform

OpenShift Container Platform

```
oc get namespace golden-images
```

Output:

| NAME          | STATUS | AGE |
|---------------|--------|-----|
| golden-images | Active | 5m  |

OpenShift Container Platform

```
oc get all -n golden-images
```

**VM** & **VMI** objects

```
oc get vm,vmi -n golden-images
```

**DataVolumes** & **PVC** objects

```
oc get dv,pvc -n golden-images
```

**DataSources** objects

```
oc get datasource -n golden-images
```

OpenShift Container Platform

□□□□ 1: □□□ VM □□□□□□

VM □□□□□□

```
oc get vm base-fedora-vm -n golden-images
```

□□□□:

| NAME           | AGE | STATUS  | READY |
|----------------|-----|---------|-------|
| base-fedora-vm | 2m  | Running | True  |

VMI □□□□□□

```
oc get vmi base-fedora-vm -n golden-images -o yaml
```

□□: - **status.phase: Running** - **status.interfaces[].ipAddress** (VM □ IP □□□□□□) - **status.guestOSInfo.id: fedora** (□□□□□□□□□□□□)

□:

```
status:
  guestOSInfo:
    id: fedora
    kernelRelease: 6.11.4-301.fc41.x86_64
    kernelVersion: '#1 SMP PREEMPT_DYNAMIC Thu Oct 10 16:16:06 UTC 2024'
    name: Fedora Linux
    prettyName: Fedora Linux 41 (Cloud Edition)
    version: '41'
    versionId: '41'
  interfaces:
  - infoSource: domain, guest-agent, multus-status
    interfaceName: eth0
    ipAddress: 10.128.0.168
    ipAddresses:
    - 10.128.0.168
    - fd02::76
    mac: 02:92:c2:00:00:06
    name: default
    queueCount: 1
  phase: Running
```

□□□ VM DataVolume □□□

```
oc get dv base-fedora-vm-disk -n golden-images
```

□□□□:

| NAME                | PHASE     | PROGRESS | RESTARTS | AGE |
|---------------------|-----------|----------|----------|-----|
| base-fedora-vm-disk | Succeeded | 100.0%   |          | 5m  |

□□□ **VM** □ **PVC** □□□

```
oc get pvc base-fedora-vm-disk -n golden-images
```

□□□□□□**Bound** □□□□□□□□□□□□□□□□:

| NAME                | STATUS       | VOLUME        | CAPACITY |
|---------------------|--------------|---------------|----------|
| ACCESS MODES        | STORAGECLASS | AGE           |          |
| base-fedora-vm-disk | Bound        | pvc-abc123... | 30Gi     |
| RWO                 | lvms-vg1     | 5m            |          |

□□□□ **2:** □□□□□□□□□□ **VM** □□□

**VM** □□□□□□□□□□□□

```
virtctl console base-fedora-vm -n golden-images
```

□□□□□□□□□□□□□□□□

**VM** □□:

```
# For nginx
systemctl status nginx
nginx -v

# For custom configuration
cat /etc/nginx/nginx.conf
ls -la /usr/share/nginx/html/
```

**Cloud-init** □□ (□□□□□□□□□□) □□□

```
cat /var/log/cloud-init.log
cat /var/log/cloud-init-output.log
```

□□□□ **3:** **VM** □□□□□□□□□□□□

**VM** □□□□□□□□□□□□

```
oc get vm base-fedora-vm -n golden-images
```

□□□□:

| NAME           | AGE | STATUS  | READY |
|----------------|-----|---------|-------|
| base-fedora-vm | 10m | Stopped | False |

**VMI** □□□□□□□□□□

```
oc get vmi base-fedora-vm -n golden-images 2>&1
```

□□□□:

Error from server (NotFound): virtualmachineinstances.kubevirt.io "base-fedora-vm" not found

□□□□ **4: DataSource** □□□□□□

**DataSource** □□□

```
oc get datasource fedora-nginx-golden -n golden-images
```

□□□□:

| NAME                | AGE |
|---------------------|-----|
| fedora-nginx-golden | 1m  |

**DataSource** □□□□□□

```
oc get datasource fedora-nginx-golden -n golden-images -o yaml
```

□□:

```
spec:
  source:
    pvc:
      name: base-fedora-vm-disk
      namespace: golden-images
```

□□□□ 5: □□□□□□□□□□□□□□□□ VM □□□

□□□□□□ VM □□□

```
oc get vm webserver-vm-1 webserver-vm-2 -n golden-images
```

□□□□:

| NAME           | AGE | STATUS  | READY |
|----------------|-----|---------|-------|
| webserver-vm-1 | 2m  | Running | True  |
| webserver-vm-2 | 2m  | Running | True  |

□□□□□□ VM □ DataVolumes □□□

```
oc get dv webserver-vm-1-disk webserver-vm-2-disk -n golden-images
```

□□□□:

| NAME                | PHASE     | PROGRESS | RESTARTS | AGE |
|---------------------|-----------|----------|----------|-----|
| webserver-vm-1-disk | Succeeded | 100.0%   |          | 2m  |
| webserver-vm-2-disk | Succeeded | 100.0%   |          | 2m  |

VM □□□□□□□□□□□□

```
oc get dv webserver-vm-1-disk -n golden-images -o jsonpath='{.spec.sourceRef}'
```

□□□□□□DataSource □□□□□□:

```
{"kind":"DataSource","name":"fedora-nginx-golden","namespace":"golden-images"}
```

VM □□□□□□

VM □ IP □□□:

```
oc get vmi webserver-vm-1 webserver-vm-2 -n golden-images -o jsonpath='{range .items[*]}{.metadata.name}: {.status.interfaces[0].ipAddress}{"\n"}{end}'
```

□□□:

```
webserver-vm-1: 10.128.0.169
webserver-vm-2: 10.128.0.170
```



Pod nginx:

```
oc run test-curl --image=curlimages/curl:latest --rm -it --restart=Never -- curl
http://10.128.0.169
```

6:

PVC

```
oc get pvc -n golden-images
```

PVC VM

| NAME                | STATUS | VOLUME        | CAPACITY | ACCESS MODES |
|---------------------|--------|---------------|----------|--------------|
| base-fedora-vm-disk | Bound  | pvc-abc123... | 30Gi     | RWO          |
| webserver-vm-1-disk | Bound  | pvc-def456... | 30Gi     | RWO          |
| webserver-vm-2-disk | Bound  | pvc-ghi789... | 30Gi     | RWO          |

(LVM)

```
# LVM
oc get lvmvolumegroup -n openshift-storage -o yaml | grep -A 10 status
```

1: **DataVolume** **ImportInProgress**

:

```
oc get dv base-fedora-vm-disk -n golden-images
NAME          PHASE          PROGRESS  AGE
base-fedora-vm-disk  ImportInProgress  10%      10m
```

:

```
# CDI importer pod
oc get pods -n golden-images | grep importer

# importer
```

```
oc logs -n golden-images <importer-pod-name>
```

□□□□□□: - □□□ DataSource □□□□□□ - □□□□□□□□□□□□□□□□□□□□□□□□ - □□□□□□□□□ - □□□□□□□□□□□□

□□□:

1. □□□ DataSource □□□□□□□□□□:

```
oc get datasource fedora -n openshift-virtualization-os-images
```

2. CDI □□□□□□□□□□□□□□:

```
oc get pods -n openshift-cnv | grep cdi
```

3. □□□□□□□□□□:

```
oc get pvc -n golden-images
oc get lvmvolumegroup -n openshift-storage
```

□□ 2: VM □□□□□□ - **DataVolume** □□□□

□□:

```
oc get vm base-fedora-vm -n golden-images
```

| NAME           | AGE | STATUS                  | READY |
|----------------|-----|-------------------------|-------|
| base-fedora-vm | 5m  | WaitingForVolumeBinding | False |

□□:

```
# DataVolume □□□□□□
oc get dv base-fedora-vm-disk -n golden-images

# □□□□□□□□□□
oc get events -n golden-images --sort-by='.lastTimestamp' | grep -i error
```

□□□□□□: - DataVolume □ **PendingPopulation** □□□□□□□□ - □□□□□□□□□□□□ - PVC □□□□□□□□□□

□□□:

1. **WaitForFirstConsumer** □□□□□□□□□□□□□□□□ - DV □□□□□□□□ VM □□□□□□
2. □□□□□□□□□□□□□□□□□□□□□□□□□□:

```
oc get storageclass
```

3. `oc get storageclass -o jsonpath='{.items[?(@.metadata.annotations.storageclass\.kubernetes\.io/is-default-class=="true")].metadata.name}'`

```
oc get storageclass -o
jsonpath='{.items[?(@.metadata.annotations.storageclass\.kubernetes\.io/is-default-
class=="true")].metadata.name}'
```

**3:** `oc get vmi base-fedora-vm -n golden-images -o jsonpath='{.status.guestOSInfo}'`

`oc:`

```
oc get vmi base-fedora-vm -n golden-images -o jsonpath='{.status.guestOSInfo}'
#
```

`oc:`

```
# VM
virtctl console base-fedora-vm -n golden-images

# qemu-guest-agent
rpm -qa | grep qemu-guest-agent

#
systemctl status qemu-guest-agent
```

`oc:`

1. `sudo dnf install -y qemu-guest-agent`

```
sudo dnf install -y qemu-guest-agent
sudo systemctl enable --now qemu-guest-agent
```

2. `oc get vmi webserver-vm-1 -n golden-images -o jsonpath='{.status.guestOSInfo}'`

**4:** `oc get vmi webserver-vm-1 -n golden-images -o jsonpath='{.status.guestOSInfo}'`

`oc:` `oc get vmi webserver-vm-1 -n golden-images -o jsonpath='{.status.guestOSInfo}'`

`oc:`

```
# VM
virtctl console webserver-vm-1 -n golden-images

# nginx
which nginx
systemctl status nginx
```



```
# PVC 삭제하기
oc get pvc -n golden-images
```

결과:

1. 사용하지 않는 PVC 삭제:

```
oc delete pvc <unused-pvc> -n golden-images
```

2. LVM 파티션 생성 (필요할 경우)
3. VM/DataVolume 파티션 생성

과제 7: VM 생성하기

결과: VM 및 DataVolume이 **Succeeded** 상태로 생성되었습니다

결과:

```
# DataVolume 생성
oc get dv webserver-vm-1-disk -n golden-images -w

# CDI 로그 확인
oc logs -n golden-images -l app=containerized-data-importer --tail=100
```

참고: - 옵션은 - 옵션과 - 옵션 (DataSource 생성)

결과:

1. DataSource 생성:

```
spec:
  sourceRef:
    kind: DataSource
    name: fedora-nginx-golden
```

2. 파티션 생성 (필요할 경우 LVM 파티션 생성)
3. VM 파티션 생성

결과

결과

```
# 확인
oc get all,dv,pvc,datasource -n golden-images

# 결과
```

```
oc get all,dv,pvc,datasource -n golden-images -o wide
```

```
# 查看
```

```
oc get vm,vmi,dv -n golden-images -w
```

查看事件

```
# 查看事件
```

```
oc get events -n golden-images --sort-by='.lastTimestamp' | tail -20
```

```
# VM 查看
```

```
oc describe vm base-fedora-vm -n golden-images | tail -20
```

```
# CDI 查看
```

```
oc logs -n openshift-cnv -l app=cdi-operator --tail=50
```

查看 PVC

```
# PVC 查看
```

```
oc describe pvc <pvc-name> -n golden-images
```

```
# 查看存储类
```

```
oc get storageclass -o yaml
```

```
# LVM 查看 (LVM 查看)
```

```
oc get lvmvolumegroup,lvmvolumegroupnodestatus -n openshift-storage
```

相关链接

- [OpenShift Virtualization 查看窗口=\\_blank](#)
- [KubeVirt 查看 - DataVolumes窗口=\\_blank](#)
- [Containerized Data Importer \(CDI\)窗口=\\_blank](#)
- [LVM 查看](#)

查看 OpenShift Virtualization 部署

部署

部署 OpenShift

Virtualization

部署 OpenShift Virtualization 部署 VM 部署

部署 VM

部署 VM

部署 OpenShift Virtualization 部署 VM 部署

部署 OpenShift Virtualization 部署 VM 部署

部署 OpenShift Virtualization 部署 VM 部署 OpenShift Virtualization 部署 2 部署 VM 部署

- **Web** コンポーネント: VM コンポーネントをインストールする
- **CLI** コンポーネント: PVC コンポーネントを VM コンポーネントに

コンポーネントのインストール:

- コンポーネントのインストール
- コンポーネントのインストール
- コンポーネントのインストール
- コンポーネントのインストール
- コンポーネントのインストール

コンポーネントのインストール

コンポーネントVM コンポーネントのインストール

| コンポーネント | コンポーネントのインストール                                 | コンポーネント                   |
|---------|------------------------------------------------|---------------------------|
| コンポーネント | コンポーネント VM コンポーネントのインストール                      | コンポーネント VM コンポーネントのインストール |
| コンポーネント | コンポーネント VM コンポーネントcloud-init<br>コンポーネントのインストール | コンポーネント VM コンポーネントのインストール |
| コンポーネント | コンポーネント VM コンポーネントのインストール                      | コンポーネントのインストール            |
| コンポーネント | コンポーネント VM コンポーネントのインストール                      | コンポーネントのインストール            |
| コンポーネント | コンポーネントのインストール                                 | コンポーネント Web コンポーネント       |

コンポーネント

- OpenShift 4.18 コンポーネント OpenShift Virtualization コンポーネントのインストール
- OpenShift Web コンポーネント CLI コンポーネント (**oc**, **virtctl**) コンポーネント
- コンポーネントのインストール
- コンポーネント VM コンポーネント 1 コンポーネントのインストール
- PersistentVolumeClaims (PVC) コンポーネントのインストール

コンポーネントのインストール

OpenShift Virtualization コンポーネントVM コンポーネントのインストール 2 コンポーネントのインストール

**Web** コンポーネントのインストール

Web コンポーネントのインストール

- PVC コンポーネントのインストール
- コンポーネントのインストール VM コンポーネント
- コンポーネント VM コンポーネントのインストール

- VM の CPU 使用率を確認する

この CLI コマンド

を実行すると、VM の CPU 使用率を確認できる。

- VM の PVC を確認する
- VM の PVC を削除する
- VM の PVC を作成する
- VM の PVC を削除する

を実行すると、

VM の PVC の使用状況を確認できる。

- 実行コマンド: `kubectl get pvc -n vm-cloning-demo`
- 実行コマンド: `kubectl delete pvc -n vm-cloning-demo`

## Web サービスの VM を作成する

Web サービスの VM を作成する。

この VM は

VM を作成する。

```
oc create namespace vm-cloning-demo
```

この VM は nginx を実行する VM を作成する。

注: この cloud-init は、VM の起動時に SSH を有効にする。

**source-webserver-vm.yaml** の内容

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: source-webserver
  namespace: vm-cloning-demo
spec:
  dataVolumeTemplates:
    - apiVersion: cdi.kubevirt.io/v1beta1
      kind: DataVolume
      metadata:
        name: source-webserver-disk
      spec:
        sourceRef:
          kind: DataSource
```



```

    name: fedora
    namespace: openshift-virtualization-os-images
  storage:
    resources:
      requests:
        storage: 30Gi
  runStrategy: RerunOnFailure
  template:
    metadata:
      labels:
        kubevirt.io/domain: source-webserver
    spec:
      domain:
        cpu:
          cores: 2
        devices:
          disks:
            - disk:
                bus: virtio
                name: rootdisk
            - disk:
                bus: virtio
                name: cloudinitdisk
          interfaces:
            - masquerade: {}
              name: default
          rng: {}
        machine:
          type: pc-q35-rhel9.4.0
        memory:
          guest: 4Gi
      networks:
        - name: default
          pod: {}
      volumes:
        - dataVolume:
            name: source-webserver-disk
            name: rootdisk
        - cloudInitNoCloud:
            userData: |-
              #cloud-config
              user: fedora
              password: fedora123
              chpasswd: { expire: false }
              ssh_pwauth: true
            runcmd:
              - dnf install -y nginx
              - systemctl enable --now nginx
              - echo "<h1>Source Webserver VM</h1><p>Hostname: $(hostname)</p>" >
                /usr/share/nginx/html/index.html
              - firewall-cmd --permanent --add-service=http

```

```
- firewall-cmd --reload
name: cloudinitdisk
```

VM を再起動

```
oc apply -f source-webserver-vm.yaml
```

VM を確認

```
oc get vmi -n vm-cloning-demo
```

**Web** VM を複製

OpenShift Web VM を複製

1. 画面 → 複製

[複製 source-webserver VM] | *clone-vm-list.png*

2. 複製 VM (**source-webserver**) を複製

3. VM を (複製) 複製

[複製 VM のアクションメニュー] | *clone-vm-actions-menu.png*

4. 複製 VM の名前:

- 複製 VM の名前: **webserver-clone-1**
- 複製 VM の CPU と NIC の設定
- 複製 VM の CPU と NIC の設定

5. 複製 VM を複製

[複製 VM のダイアログ] | *clone-vm-dialog.png*

Web VM を複製

- 複製 VM の PVC を複製
- 複製 VM の CPU と NIC の設定
- VM の CPU と NIC の設定
- 複製 VM の CPU と NIC の設定

複製 VM の PVC を複製 "複製" を複製

[複製 VM のダイアログ] | *clone-vm-dialog-full.png*

## Web ブラウザからアクセス

ブラウザから VM にアクセスする

```
oc get vm -n vm-cloning-demo
oc get vmi -n vm-cloning-demo
```

ブラウザから VM にアクセスする

```
virtctl console webserver-clone-1 -n vm-cloning-demo
```

ブラウザから VM に nginx (ポート 80) をインストールする

## CLI からインストールする

インストールする前に、CLI からインストールする

ステップ 1: PVC を作成する VM を作成する

CLI からインストールする VM の PVC を作成する

```
oc get pvc -n vm-cloning-demo
```

インストールする

| NAME                  | STATUS   | VOLUME                                   | CAPACITY |
|-----------------------|----------|------------------------------------------|----------|
| source-webserver-disk | Bound    | pvc-1234abcd-5678-efgh-9012-ijklmnop3456 | 30Gi     |
| RWO                   | lvms-vg1 | 10m                                      |          |

ステップ 2: PVC をインストールする DataVolume を作成する

clone-datavolume.yaml をインストール

```
apiVersion: cdi.kubevirt.io/v1beta1
kind: DataVolume
metadata:
  name: webserver-clone-2-disk
  namespace: vm-cloning-demo
  annotations:
    cdi.kubevirt.io/storage.bind.immediate.requested: "true" ①
spec:
  source:
    pvc:
      name: source-webserver-disk
      namespace: vm-cloning-demo
```

```
storage:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 30Gi
  storageClassName: gp3-csi ②
```

① `WaitForFirstConsumer` ②

② PVC

`cdi.kubevirt.io/storage.bind.immediate.requested: "true"` `WaitForFirstConsumer` (AWS EBS gp3-csi Azure Disk GCP PD) `DataVolume` `PendingPopulation` `DataVolume` VM `PendingPopulation`

`DataVolume`

```
oc apply -f clone-datavolume.yaml
```

```
oc get dv webserver-clone-2-disk -n vm-cloning-demo -w
```

| NAME                   | PHASE                             | PROGRESS | RESTARTS | AGE |
|------------------------|-----------------------------------|----------|----------|-----|
| webserver-clone-2-disk | CloneFromSnapshotSourceInProgress | N/A      |          | 4s  |
| webserver-clone-2-disk | PrepClaimInProgress               | N/A      |          | 68s |
| webserver-clone-2-disk | RebindInProgress                  | N/A      |          | 78s |
| webserver-clone-2-disk | Succeeded                         | 100.0%   |          | 79s |

`DataVolume` `Succeeded` VM

`vm-from-cloned-pvc.yaml`

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: webserver-clone-2
  namespace: vm-cloning-demo
spec:
  runStrategy: Manual
  template:
    metadata:
      labels:
        kubevirt.io/domain: webserver-clone-2
```

```
spec:
  domain:
    cpu:
      cores: 2
    devices:
      disks:
        - disk:
            bus: virtio
            name: rootdisk
      interfaces:
        - masquerade: {}
          name: default
      rng: {}
    machine:
      type: pc-q35-rhel9.4.0
    memory:
      guest: 4Gi
    networks:
      - name: default
        pod: {}
    volumes:
      - name: rootdisk
        persistentVolumeClaim:
          claimName: webserver-clone-2-disk
```

VM を適用

```
oc apply -f vm-from-cloned-pvc.yaml
```

VM を起動

```
virtctl start webserver-clone-2 -n vm-cloning-demo
```

ステップ 2: VM を確認

ステップ 1: VM を確認

ステップ 1: VM を確認

```
virtctl stop source-webserver -n vm-cloning-demo
```

VM を確認

```
oc get vm source-webserver -n vm-cloning-demo
```

ステップ 1: VM を確認

```
oc get vm source-webserver -n vm-cloning-demo -o yaml > clone-base.yaml
```

clone-base.yaml

### 1. VM

```
metadata:
  name: source-webserver # : webserver-manual-clone
  namespace: vm-cloning-demo
```

### 2. DataVolume PVC

```
spec:
  dataVolumeTemplates:
    - apiVersion: cdi.kubevirt.io/v1beta1
      kind: DataVolume
      metadata:
        name: source-webserver-disk # : webserver-manual-clone-disk
      spec:
        # sourceRef
        # :
        source:
          pvc:
            name: source-webserver-disk
            namespace: vm-cloning-demo
        storage:
          resources:
            requests:
              storage: 30Gi
```

### 3.

```
spec:
  template:
    metadata:
      labels:
        kubevirt.io/domain: source-webserver # : webserver-manual-clone
    spec:
      domain:
        # ...
      volumes:
        - dataVolume:
            name: source-webserver-disk # : webserver-manual-clone-disk
            name: rootdisk
```

### 4.

metadata.uid

- metadata.resourceVersion
- metadata.creationTimestamp
- metadata.generation
- status

## 5. MAC

macAddress

```
spec:
  template:
    spec:
      domain:
        devices:
          interfaces:
            - masquerade: {}
              name: default
              macAddress: "52:54:00:xx:xx:xx" #
```

MAC Failed to allocate mac to the vm object: failed to allocate requested mac address

YAML

```
oc apply -f clone-base.yaml
```

2 DataVolume

```
cat <<EOF | oc apply -f -
apiVersion: cdi.kubevirt.io/v1beta1
kind: DataVolume
metadata:
  name: webserver-custom-disk
  namespace: vm-cloning-demo
  annotations:
    cdi.kubevirt.io/storage.bind.immediate.requested: "true"
spec:
  source:
```

```
pvc:
  name: source-webserver-disk
  namespace: vm-cloning-demo
storage:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 30Gi
EOF
```

```
oc get dv webserver-custom-disk -n vm-cloning-demo -w
```

## cloned-vm-with-cloudinit.yaml

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: webserver-custom
  namespace: vm-cloning-demo
spec:
  runStrategy: Manual
  template:
    metadata:
      labels:
        kubevirt.io/domain: webserver-custom
    spec:
      domain:
        cpu:
          cores: 2
        devices:
          disks:
            - disk:
                bus: virtio
                name: rootdisk
            - disk:
                bus: virtio
                name: cloudinitdisk
          interfaces:
            - masquerade: {}
              name: default
          rng: {}
        machine:
          type: pc-q35-rhel9.4.0
        memory:
```



```

    guest: 4Gi
networks:
  - name: default
    pod: {}
volumes:
  - name: rootdisk
    persistentVolumeClaim:
      claimName: webserver-custom-disk
  - cloudInitNoCloud:
      userData: |-
        #cloud-config
        hostname: webserver-custom
        fqdn: webserver-custom.local
        manage_etc_hosts: true
        password: fedora123
        chpasswd: { expire: false }
        ssh_pwauth: true
      runcmd:
        - hostnamectl set-hostname webserver-custom
        - echo "<h1>Customized Clone</h1><p>Hostname: $(hostname)</p>" >
          /usr/share/nginx/html/index.html
        - systemctl restart nginx
    name: cloudinitdisk

```

□□□□□□□□□□□□□□□□:

- □□□□□ **webserver-custom** □□□□□□
- **fedora** □□□□□□□□□□□□□□□□
- **nginx** □□□□□□□□□□□□
- **nginx** □□□□□□□□□□□□□□□□

□□□□□□□□□□

```

oc apply -f cloned-vm-with-cloudinit.yaml
virtctl start webserver-custom -n vm-cloning-demo

```

□□□□□□□□□□□□

```

virtctl console webserver-custom -n vm-cloning-demo

```

VM □□:

```

hostname
# □□: webserver-custom

systemctl status nginx

```

```
# nginx 실행 (컨테이너) 실행
```

```
curl localhost
```

```
# 실행 HTML 컨테이너
```

컨테이너 실행

컨테이너 실행 컨테이너 실행 컨테이너 실행 VM 컨테이너 실행

컨테이너 실행

```
oc create namespace vm-cloning-production
```

컨테이너 실행 RBAC 실행

컨테이너 실행 컨테이너 실행 PVC 컨테이너 실행 RBAC 컨테이너 실행

[cross-namespaced-clone-rbac.yaml](#) 컨테이너 실행

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: datavolume-cloner
  namespace: vm-cloning-demo
rules:
  - apiGroups: [""]
    resources: ["persistentvolumeclaims"]
    verbs: ["get", "list"]
  - apiGroups: ["cdi.kubevirt.io"]
    resources: ["datavolumes"]
    verbs: ["get", "list"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: allow-clone-from-demo
  namespace: vm-cloning-demo
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: datavolume-cloner
subjects:
  - kind: ServiceAccount
    name: default
    namespace: vm-cloning-production
```

RBAC 컨테이너 실행

```
oc apply -f cross-namespace-clone-rbac.yaml
```

□□□□□□□□□□

□□□□□□□□□□□□ DataVolume □□□□□□□□□□□□□□□□□□

**cross-namespace-clone-datavolume.yaml** □□□□□□□

```
apiVersion: cdi.kubevirt.io/v1beta1
kind: DataVolume
metadata:
  name: webserver-production-disk
  namespace: vm-cloning-production
  annotations:
    cdi.kubevirt.io/storage.bind.immediate.requested: "true"
spec:
  source:
    pvc:
      name: source-webserver-disk
      namespace: vm-cloning-demo
  storage:
    accessModes:
      - ReadWriteOnce
    resources:
      requests:
        storage: 30Gi
```

DataVolume □□□□□□

```
oc apply -f cross-namespace-clone-datavolume.yaml
```

□□□□□□□□□□□□

```
oc get dv -n vm-cloning-production -w
```

□□□□□□□ VM □□□□□□□

**cloned-webserver-production.yaml** □□□□□□□

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: webserver-production
  namespace: vm-cloning-production
  labels:
    environment: production
```

```

spec:
  runStrategy: Manual
  template:
    metadata:
      labels:
        kubevirt.io/domain: webserver-production
    spec:
      domain:
        cpu:
          cores: 4
        devices:
          disks:
            - disk:
                bus: virtio
                name: rootdisk
            - disk:
                bus: virtio
                name: cloudinitdisk
          interfaces:
            - masquerade: {}
              name: default
          rng: {}
        machine:
          type: pc-q35-rhel9.4.0
        memory:
          guest: 8Gi
        networks:
          - name: default
            pod: {}
        volumes:
          - name: rootdisk
            persistentVolumeClaim:
              claimName: webserver-production-disk
          - cloudInitNoCloud:
              userData: |-
                #cloud-config
                hostname: webserver-prod
                fqdn: webserver-prod.production.local
                manage_etc_hosts: true
                password: fedora123
                chpasswd: { expire: false }
                ssh_pwauth: true
              runcmd:
                - hostnamectl set-hostname webserver-prod
                - echo "<h1>Production Webserver</h1><p>Environment: Production</p>" >
                  /usr/share/nginx/html/index.html
                - systemctl restart nginx
            name: cloudinitdisk

```

□□□□□□□□□□□□:

- 100GB
- 4 8Gi RAM
- 100GB
- 100GB
- 100GB

100GB

```
oc apply -f cloned-webserver-production.yaml
virtctl start webserver-production -n vm-cloning-production
```

100GB

100GB

100GB

- CSI: Ceph RBD, AWS EBS, Azure Disk, GCP PD
  - SnapshotForSmartCloneInProgress → SmartClonePVCInProgress
- CloneInProgress: CloneInProgress
  - CloneInProgress
  - CloneInProgress

100GB

```
oc get dv <datavolume-name> -n <namespace> -o yaml | grep cloneType
```

100GB:

```
cdi.kubevirt.io/cloneType: snapshot # 100GB
# 100GB
cdi.kubevirt.io/cloneType: copy # 100GB
```

100GB

100GB

- 100GB
- 100GB
- 100GB: (100GB × 100GB)

- 確認する

確認する

```
oc get sc
oc describe sc <storage-class-name>
```

確認する

- 確認する
- 確認する
- 確認する
- 確認する
- 確認する

確認する

確認する

DataVolume 確認する

```
oc get dv -n vm-cloning-demo
```

確認する **Succeeded** 確認する

| NAME                  | PHASE     | PROGRESS | AGE |
|-----------------------|-----------|----------|-----|
| webserver-custom-disk | Succeeded | N/A      | 5m  |

確認する

確認する

```
oc describe dv <datavolume-name> -n <namespace>
```

確認する

```
Events:
  Type    Reason             Age   From                                     Message
  ----    -
  Normal  CloneInProgress    5m    datavolume-controller                  Cloning from vm-cloning-
demo/source-webserver-disk in progress
  Normal  CloneSucceeded     2m    datavolume-controller                  Successfully cloned from vm-
```

```
cloning-demo/source-webserver-disk
```

VM 创建成功

```
oc get vmi -n vm-cloning-demo
```

创建成功

VM 创建成功

```
virtctl console <vm-name> -n <namespace>
```

VM 中:

```
# 查看主机名
hostname

# 查看 VM 内部系统状态
systemctl status nginx

# cloud-init 是否成功
cloud-init status

# cloud-init 输出日志
sudo cat /var/log/cloud-init-output.log

# 查看虚拟机 ID
cat /etc/machine-id
```

创建成功

状态: 从 **"Provisioning"** 变为 **"CloneInProgress"** 表示正在克隆

状态: DataVolume 处于 **CloneInProgress** 状态表示正在克隆

注意: \* 克隆过程中 \* 克隆过程中 \* 克隆过程中

状态:

```
# 查看 DataVolume 状态
oc describe dv <datavolume-name> -n <namespace>

# CDI 状态
oc get pods -n <namespace> | grep cdi
```

```
# CDI
oc logs <cdi-pod> -n <namespace>
```

~~~~~

□□: □□□□□□ **"permission denied"** □□□□□□

□□: □□□□□□□□□□ RBAC □□□□□□□□□□

□□□: RBAC □□□□□□□□□□

```
#
oc get role datavolume-cloner -n <source-namespace>

:!navtitle:

:docname: vm-snapshots
:page-module: vm-lifecycle
:page-relative-src-path: vm-snapshots.adoc
:page-origin-url: https://github.com/RedHatQuickCourses/ocp-virt-cookbook_ja
:page-origin-start-path:
:page-origin-refname: main
:page-origin-reftype: branch
:page-origin-refhash: (worktree)
:navtitle: VM
[#vm-lifecycle:vm-snapshots]
=== VM

[discrete#vm-lifecycle:vm-snapshots::_]
====

VM
~~~~~

~~~~~OpenShift Virtualization ~ VM

[discrete#vm-lifecycle:vm-snapshots::_]
====

* VM
* Web CLI
* VM
*

~~~~~:
[source,text]
```



```
[discrete#vm-lifecycle:vm-snapshots::_]
====

* OpenShift 4.18+  OpenShift Virtualization
* VM
* CSI
* 'oc'  'virtctl' CLI

[discrete#vm-lifecycle:vm-snapshots::_]
====

CSI:

[source,bash,role=execute]
```

oc get volumesnapshotclass

```
:
[source,text]
```

NAME DRIVER DELETIONPOLICY AGE csi-hostpath-snapclass hostpath.csi.k8s.io Delete 30d ocs-storagecluster-rbdplugin-snapclass openshift-storage.rbd.csi.ceph.com Delete 30d

```
VolumeSnapshotClass  VolumeSnapshotClass

[discrete#vm-lifecycle:vm-snapshots::_vm_]
==== VM

[discrete#vm-lifecycle:vm-snapshots::_]
====

OpenShift Virtualization 2 VM:

**VirtualMachineSnapshot**::
VM VM

**VirtualMachineSnapshotContent**::
VirtualMachineSnapshot VolumeSnapshot

[discrete#vm-lifecycle:vm-snapshots::_]
====

[cols="1,2,2",options="header"]
|==
|  |  |
|
| VM
```


[illegible]

```
apiVersion: snapshot.kubevirt.io/v1beta1 kind: VirtualMachineSnapshot metadata: name: database-vm-snapshot-01 namespace: vm-snapshots spec: source: apiGroup: kubevirt.io kind: VirtualMachine name: database-vm
```

```
[source,bash,role=execute]
```

```
oc apply -f vm-snapshot.yaml
```

```
[discrete#vm-lifecycle:vm-snapshots::_0000000000000000]
===== 0000000000000000

000000000000000000000000:

[source,bash,role=execute]
```

```
oc get virtualmachinesnapshot -n vm-snapshots
```

```
####:
[source,text]
```

```
NAME SOURCEKIND SOURCENAME PHASE READYTOUSE CREATIONTIME ERROR database-vm-
snapshot-20260304-155755 VirtualMachine database-vm Succeeded true 8m
```

```
#####:

[source,bash,role=execute]
```

oc describe virtualmachinesnapshot <snapshot-name> -n vm-snapshots

```
#####:

* **Phase**: `InProgress`, `Succeeded`,  `Failed`
* **ReadyToUse**: `true` ##### (Web ##### **Ready** #####)
* **Error**: #####

[discrete#vm-lifecycle:vm-snapshots::_#####]
==== #####

[discrete#vm-lifecycle:vm-snapshots::_#####]
===== #####

* **#####**:

[source,bash,role=execute]
```

oc get virtualmachinesnapshot -n vm-snapshots

```
* **#####**:

[source,bash,role=execute]
```

oc get virtualmachinesnapshot -A

```
* **Web #####**:

* VM #####
* **#####** #####
* #####

image::vm-snapshots/snapshot-tab-ready.png[#####]

[discrete#vm-lifecycle:vm-snapshots::_#####]
```

```
=====
```

```
[source,bash,role=execute]
```

oc describe virtualmachinesnapshot <snapshot-name> -n vm-snapshots

```
VolumeSnapshot
```

```
[source,bash,role=execute]
```

oc get virtualmachinesnapshotcontent -n vm-snapshots

```
:
```

```
[source,text]
```

```
NAME      READYTOUSE  CREATIONTIME  ERROR  vmsnapshot-content-cd166db8-00be-4aa9-972e-dea669527bfd  true  8m
```

```
[source,bash,role=execute]
```

oc get volumesnapshot -n vm-snapshots

```
:
```

```
[source,text]
```

```
NAME      READYTOUSE  SOURCEPVC  RESTORESIZE  SNAPSHOTCLASS  CREATIONTIME  AGE  lvms-vg1  8m  8m
vmsnapshot-cd166db8-00be-4aa9-972e-dea669527bfd-volume-rootdisk  true  database-vm-disk  30Gi
```

```
[discrete#vm-lifecycle:vm-snapshots::_]
```

```
=====
```

```
VM VM VM
```

```
targetReadinessPolicy: StopTarget`
VirtualMachineRestore spec `FailImmediate`
`WaitGracePeriod` ( 5 )`WaitEventually` ( )
```

```
link:https://docs.redhat.com/en/documentation/openshift_container_platform/1
atest/html/virtualization/backup-and-restore[Red Hat ]
```

```
[discrete#vm-lifecycle:vm-snapshots::_web_]
```

```
===== Web
```

```
* VM
```

```
* VM ( )
```

```

* **Database** Database
* Database
* **VirtualMachineSnapshot** Database
+
image::vm-snapshots/snapshot-kebab-restore.png[Database (Database)]
+
* Database
+
image::vm-snapshots/snapshot-restore-confirm.png[Database]

[discrete#vm-lifecycle:vm-snapshots::_cli_]
===== CLI

VirtualMachineRestore Database:

link:attachment$vm-snapshots/vm-restore.yaml[vm-restore.yaml Database,role=unresolved]

[source,yaml]

```

apiVersion: snapshot.kubevirt.io/v1beta1 kind: VirtualMachineRestore metadata: name: database-vm-restore-01 namespace: vm-snapshots spec: target: apiGroup: kubevirt.io kind: VirtualMachine name: database-vm virtualMachineSnapshotName: database-vm-snapshot-01

```

Database:

[source,bash,role=execute]

```

oc apply -f vm-restore.yaml

```

[discrete#vm-lifecycle:vm-snapshots::_]
=====

Database:

[source,bash,role=execute]

```

oc get virtualmachinerestore -n vm-snapshots

```

:
[source,text]

```

NAME	TARGETKIND	TARGETNAME	COMPLETE	RESTORETIME	ERROR	database-vm-restore-01
VirtualMachine	database-vm	true	2m			

```

Database:

```

```
[source,bash,role=execute]
```

oc describe virtualmachinerestore database-vm-restore-01 -n vm-snapshots

```
#####VM #####:
```

```
[source,bash,role=execute]
```

virtctl start database-vm -n vm-snapshots

```
VM #####:
```

```
[source,bash,role=execute]
```

oc get vm database-vm -n vm-snapshots

```
[discrete#vm-lifecycle:vm-snapshots::_#####]
```

```
==== #####
```

```
[discrete#vm-lifecycle:vm-snapshots::_web_#####]
```

```
===== Web #####
```

```
* VM   **#####** #####
```

```
* #####
```

```
* **#####** #####
```

```
+
```

```
image::vm-snapshots/snapshot-kebab-delete.png[##### (#####)]
```

```
+
```

```
* #####
```

```
+
```

```
image::vm-snapshots/snapshot-delete-confirm.png[VirtualMachineSnapshot #####]
```

```
[discrete#vm-lifecycle:vm-snapshots::_cli_#####]
```

```
===== CLI #####
```

```
[source,bash,role=execute]
```

oc delete virtualmachinesnapshot database-vm-snapshot-01 -n vm-snapshots

```
?: VirtualMachineSnapshot ##### VirtualMachineSnapshotContent ##### VolumeSnapshots  
#####
```

```
[discrete#vm-lifecycle:vm-snapshots::_#####]
```

```
==== #####
```

```
[discrete#vm-lifecycle:vm-snapshots::_csi_#####]
```

===== CSI {{{{

VM {{{{ CSI ({{{) {{{:

- * Red Hat OpenShift Data Foundation (ODF)
- * AWS EBS CSI
- * Microsoft Azure Disk CSI
- * Google Cloud Persistent Disk CSI
- * LVM Storage (LVMS)thin-pool {{{
- * VMware vSphere CSI
- * NetApp Trident
- * Pure Storage
- * Dell CSI {{{

[discrete#vm-lifecycle:vm-snapshots::_volumesnapshotclass_{{}}

===== VolumeSnapshotClass {{{

VolumeSnapshotClass {{{ VolumeSnapshotClass {{{:

[source,bash,role=execute]



oc get storageclass

VolumeSnapshotClass

oc get volumesnapshotclass

```
snapshot class
[discrete#vm-lifecycle:vm-snapshots::_]
=====
:
* **CSI**: CSI
* **VM**: VM
* **:
* **:
[source,bash,role=execute]
```

oc get volumesnapshot -n vm-snapshots -o wide

```
:
[source,text]
```

NAME	READYTOUSE	SOURCEPVC	RESTORESIZ	SNAPSHOTCLASS	CREATIONTIME	AGE
vmnsnapshot-cd166db8-00be-4aa9-972e-dea669527bfd-volume-rootdisk	true	database-vm-disk	30Gi	lvms-vg1	9m	9m

```
[discrete#vm-lifecycle:vm-snapshots::_]
=====

[discrete#vm-lifecycle:vm-snapshots::_inprogress_]
===== InProgress

**VirtualMachineSnapshot `InProgress`

**::

* (5 `failureDeadline`)
* QEMU
*

**::

[source,bash,role=execute]
```

oc describe virtualmachinesnapshot <snapshot-name> -n <namespace>

```
`status.conditions` 与 `status.indications` 是快照 spec 中  
`failureDeadline` 的失败截止时间
```

```
[discrete#vm-lifecycle:vm-snapshots::_noquestagent_QUIESCEFAILED_]  
=====  NoGuestAgent   QUIESCEFAILED
```

```
**:**:  `NoGuestAgent`  `QUIESCEFAILED`
```

```
**:**: VM  QEMU
```

```
[source,bash]
```

Fedora/RHEL VM 快照

```
sudo dnf install -y qemu-guest-agent sudo systemctl enable --now qemu-guest-agent
```

```
#####:
```

```
[source,bash,role=execute]
```

```
oc get vmi database-vm -n vm-snapshots -o jsonpath='{.status.conditions}' | jq '.[] | select(.type=="AgentConnected")'
```

```
[discrete#vm-lifecycle:vm-snapshots::_#####vm_#####]
```

```
===== #####VM #####
```

```
**:** VirtualMachineRestore ##### VM #####
```

```
**:**: ##### VM #####VirtualMachineRestore spec | `targetReadinessPolicy: StopTarget`  
#####:
```

```
[source,bash,role=execute]
```

```
virtctl stop database-vm -n vm-snapshots oc wait vm database-vm -n vm-snapshots  
--for=jsonpath='{.status.printableStatus}'=Stopped --timeout=120s
```

```
[discrete#vm-lifecycle:vm-snapshots::_#####volumesnapshotclass_#]
```

```
===== #####VolumeSnapshotClass #
```

```
**:**: ##### VolumeSnapshotClass #####
```

```
**:**: ##### VolumeSnapshotClass #####:
```

```
[source,bash,role=execute]
```

```
oc get storageclass -o jsonpath='{range .items[]}{.metadata.name}{"\t"}{.provisioner}{"\n"}{end}'  
oc get volumesnapshotclass -o jsonpath='{range .items[]}{.metadata.name}{"\t"}{.driver}{"\n"}{end}'
```

```
VolumeSnapshotClass | `driver` ##StorageClass | `provisioner`
```

```
#####VolumeSnapshotClass #####
```

```
[discrete#vm-lifecycle:vm-snapshots::_#####]
```

```
===== #####
```

```
#####:
```

```
[source,bash,role=execute]
```

```
oc delete virtualmachinerestore --all -n vm-snapshots oc delete virtualmachinesnapshot --all -n vm-snapshots
```

```
#####:
```

```
[source,bash,role=execute]
```

```
oc get virtualmachinesnapshot -n vm-snapshots oc get volumesnapshot -n vm-snapshots
```

```
[discrete#vm-lifecycle:vm-snapshots::_]
```

```
====
```

```
#####:
```

```
* OpenShift Virtualization VM #####
* #####
* Web CLI #####
* #####
* VM #####
* #####
* #####
```

```
[discrete#vm-lifecycle:vm-snapshots::_]
```

```
====
```

```
* xref:getting-started:virtctl-basics[] - VM CLI
* xref:vm-lifecycle:custom-golden-images[] - VM
* xref:vm-lifecycle:cloning-vms[] - VM (())
*
```

```
link:https://docs.redhat.com/en/documentation/openshift\_container\_platform/latest/html/virtualization/backup-and-restore[Red Hat - VM
```

```
#####,window=_blank]
```

```
* link:https://kubevirt.io/user-guide/storage/snapshot\_restore\_api/[KubeVirt -
##### API,window=_blank]
```

```
:!navtitle:
```

```
:docname: boot-order-boot-sources
```

```
:page-module: vm-lifecycle
```

```
:page-relative-src-path: boot-order-boot-sources.adoc
```

```
:page-origin-url: https://github.com/RedHatQuickCourses/ocp-virt-cookbook\_ja
```

```
:page-origin-start-path:
```

```
:page-origin-refname: main
```

```
:page-origin-reftype: branch
```

```
:page-origin-refhash: (worktree)
```

```
:navtitle: VM #####
```

```
[#vm-lifecycle:boot-order-boot-sources]
```


apiVersion: kubevirt.io/v1

#

1. 了解 LVM 和 Kubernetes 的集成

##

操作	命令	说明
<code>[manifests:index:::00000]</code>	7	LVM 部署和配置
<code>[manifests:index:::00000]</code>	8	配置 UDN 和存储类
<code>[manifests:index:::00000]</code>	14	配置 VM 和存储类
<code>[manifests:index:::00000]</code>	3	VM 和 Kubernetes 集成

##

LVM 部署和配置

名称	类型	描述	部署方法	文件
lvm-namespace	Namespace	openshift-lvm-storage 命名空间	LVM Operator 部署	YAML
lvm-operatorgroup	Operator Group	LVM 部署和配置	LVM Operator 部署	YAML
lvm-subscription	Subscription	OperatorHub 上 LVM Storage Operator 订阅	LVM Operator 部署	YAML
lvmcluster-with-wipe	LVMCluster	forceWipe 配置 LVM 存储类	LVM Operator 部署	YAML
lvmcluster-clean-disk	LVMCluster	配置 LVM 存储类	LVM Operator 部署	YAML
test-pvc	PVC	配置 PVC 存储类	LVM Operator 部署	YAML
test-pod	Pod	PVC 配置 Pod 存储类	LVM Operator 部署	YAML

##

配置 UDN 和存储类

이름	네임스페이스	설명	필요한 라이브러리	파일 형식
localnet-secondary-namespace	Namespace	localnet 2 VM 네임스페이스	(CUDN) OpenShift Virtualization localnet 2 네임스페이스	YAML
localnet-bridge-mapping-nncp	NNCP	localnet1 브리지 br-ex localnet 네임스페이스	(CUDN) OpenShift Virtualization localnet 2 네임스페이스	YAML
cudn-localnet	CUDN	localnet 2 ClusterUserDefinedNetwork	(CUDN) OpenShift Virtualization localnet 2 네임스페이스	YAML
localnet-vlan-namespace	Namespace	VLAN localnet VM 네임스페이스	(CUDN) OpenShift Virtualization VLAN 네임스페이스 NetworkAttachmentDefinition OpenShift Virtualization 네임스페이스	YAML
localnet-vlan-bridge-mapping-nncp	NNCP	localnet-vlan 브리지 br-ex VLAN 네임스페이스	(CUDN) OpenShift Virtualization VLAN 네임스페이스 NetworkAttachmentDefinition OpenShift Virtualization 네임스페이스	YAML
localnet-vlan-100-nad	NAD	VLAN 100 NetworkAttachmentDefinition	(CUDN) OpenShift Virtualization VLAN 네임스페이스 NetworkAttachmentDefinition OpenShift Virtualization 네임스페이스	YAML
udn-primary-namespace	Namespace	UDN 네임스페이스	(UDN) OpenShift Virtualization localnet 2 네임스페이스	YAML
udn-primary	UDN	VM Layer 2 UserDefinedNetwork	(UDN) OpenShift Virtualization localnet 2 네임스페이스	YAML

□□□□□

□□□□□□□□□□ VM □□□□□□□□□□□□□□□□

□□	□□	□□	□□□□□□□	□□□□□□
fedora-localnet-vm	VirtualMachine	localnet 2 □□□□□□□□□□ Fedora VM	□□□□□□□□□□□□□□ (CUDN) □□□□ OpenShift Virtualization □□□□□□□ 2 □□□□□□□□□□	YAML
fedora-vlan-vm	VirtualMachine	VLAN 100 □□ secondary network □□□ Fedora VM	□□□□□□□□□□□□□□□□ □□□□ VLAN □□□ NetworkAttachment Definition □□□□ OpenShift Virtualization □□□	YAML
fedora-vm-with-udn	VirtualMachine	UDN □□□□□□□□□□□□□□□□□□ Fedora VM	□□□□□□□□□□□ (UDN) □□□□□□□□ 2 □□□□□□□□□□□□□□	YAML
fedora-dhcp-vm	VirtualMachine	DHCP □□□□□□□□□□□□□□□□□□ Fedora VM	□□IP□□□□□□□□	YAML
fedora-multi-network-vm	VirtualMachine	DHCP □□□□□□ + □□ IP 2 □□□□□□□□□□□□□□□□□□ Fedora VM	□□IP□□□□□□□□	YAML
fedora-vlan-static-ip-vm	VirtualMachine	VLAN □□□□□□□□□□□□ IP □□□ Fedora VM	□□IP□□□□□□□□	YAML
golden-image-base-vm	VirtualMachine	□□□□□□□□□□□□□□□□□□ VM	□□□□□□□□□□	YAML
webserver-vm-from-golden	VirtualMachine	□□□□□□□□□□□□□□□□□□ VM	□□□□□□□□□□	YAML
webserver-vm-2	VirtualMachine	□□□□□□□□□□□□ 2 □□□□□□□□□□ VM	□□□□□□□□□□	YAML
fedora-access-vm	VirtualMachine	SSH □□□□□□□□□□□□□□□□□□ Fedora VM	□□□□□□□□□□□□Linux VM□□□□□□□	YAML
fedora-vlan-access-vm	VirtualMachine	VLAN □□□□□□□□□□ SSH □□□□□□□□□□ Fedora VM	□□□□□□□□□□□□Linux VM□□□□□□□	YAML
fedora-app-vm	VirtualMachine	□□ DNS □□□□□□□□□□□□□□□□□□ Fedora VM	□□□□□□□□□□ DNS □□	YAML
golden-image-datasource	DataSource	□□□□□□□□□□ PVC □□□□ DataSource	□□□□□□□□□□	YAML

API

OpenShift Virtualization API

OpenShift Virtualization OpenShift Container Platform

- API
- API
- API
- API
- API
- API
- VM

- OpenShift v4.18+ OpenShift Virtualization
- kubeadmin
- REST API HTTP CRUD GETPUTPOSTPATCHDELETEJSON
- REST API PostmancURL

API

OpenShift Virtualization API

OpenShift Virtualization

- [KubeVirt API](#)
- [KubeVirt](#)

API

OpenShift v4.19, v4.20

OpenShift API

OpenShift API (Kubernetes) OpenShift (Kubernetes) API json yaml OpenShift API

API API (API)

yaml https://docs.redhat.com/en/documentation/openshift_container_platform/latest/html/virtualization/index[OpenShift,window=_blank] KubeVirt API https://kubevirt.io/api-reference/main/index.html[KubeVirt API,window=_blank]

- [API](#)
 -
 -
 -
 -
 -
 -
 -
 -
 -
- [API](#)
 -
 -
 -
 -

- `NMState` オブジェクト
- **API**
 - `API`
 - `API`
 - `API`
 - `API`
 - `API`
 - `API`
 - `API`
 - `API`
- **API**
 - `API`
 - `API`
 - `API`
 - `API`
- **API**
 - `API`

API

API

API (Kubernetes) OpenShift

() Status

`spec` VM:

- `instanceType` - (RAM CPU)
- `volumes` - VM https://kubevirt.io/user-guide/storage/disks_and_volumes/#volumes `dataVolume` `persistentVolumeClaims`
 - `dataVolume` - `dataVolumeTemplates`
 - `persistentVolumeClaim` - VM (PVC)
- `networks` - VM Pod/VM
- `interfaces` - VM NIC Pod/VM
- `dataVolumeTemplates` - `volumes`

spec.runStrategy によって VM の実行戦略が決定されます:

- **Always** - VM が停止したとき (何らかの理由で) 自動的に VM を再起動します。
- **RerunOnFailure** - Always と同じですが、VM が失敗したときのみ再起動します。
- **Once** - VM が 1 回実行された後、自動的に停止します。
- **Manual** - VM が POST された後、自動的に実行されません。
- **Halted** - VM が spec.runStrategy によって VM が start された後 (自動的に) 自動的に停止します。

spec.running によって spec.runStrategy によって VM の実行戦略が決定されます。

virtctl start|stop|restart によって VM の status (KubeVirt の VM の status) API によって決定されます。

spec.runStrategy によって VM の実行戦略が決定されます:

- **start** - VM が Halted 状態にある場合、Always、RerunOnFailure、Manual によって自動的に VM を起動します。
- **restart** - VM が Halted 状態にある場合、VM を自動的に再起動します。
- **stop** - VM が Always、Halted、RerunOnFailure、Manual によって自動的に VM を停止します。

API によって stateChangeRequests[] によって status によって決定されます:

```
status:
  stateChangeRequests:
    - action: <Stop|Start> ①
      uid: <uid-of-the-virtual-machine-instance> ②
```

① Stop または Start によって自動的に VM を再起動します。

② 自動的に VM を再起動する場合 (自動的に) は、UID によって自動的に VM を再起動します。 VMI によって自動的に再起動します。

spec.status によって VM の status によって決定されます:

- **status.printableStatus** - VM の status (Running (成功/失敗) Starting (成功/失敗) Stopped) によって決定されます。
- **status.runStrategy** - spec.runStrategy によって決定されます (Always、RerunOnFailure、Once、Manual、Halted) によって決定されます。
- **status.conditions["type": "Ready"]** - conditions によって Ready (成功) によって決定されます。 status - Reason Message - Ready によって決定されます "Ready" によって決定されます。

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: vm-status-example
spec:
```



```

...
status:
  conditions:
    - lastProbeTime: "2025-11-10T15:00:46Z"
      lastTransitionTime: "2025-11-10T15:00:46Z"
      message: Guest VM is not reported as running
      reason: GuestNotRunning
      status: "False"
      type: Ready

```

OpenShift

OpenShift (VMI) OpenShift VM

VMI Kubernetes Pod OpenShift Pod (OpenShift) QEMU / KVM OpenShift Pod OpenShiftVM OpenShift (OpenShift) PersistentVolumeClaim OpenShift

OpenShiftVM yml (json) template spec.template OpenShiftVM VMI OpenShift

VM OpenShiftVMI OpenShift (DataVolumePVC) OpenShift

VM OpenShiftVM OpenShift VMI OpenShift VMI OpenShift

OpenShift

OpenShift CPU OpenShift (u1) OpenShift (xxlarge2xlarge) OpenShift (u1.xlarge) OpenShift CPU OpenShift

OpenShift OpenShift:

- **(N)** - OpenShiftDPDK enabled OpenShift1:2 CPU/RAM
- **(O)** - RAM OpenShift1:4 CPU/RAM
- **(CX)** - CPU OpenShift1:2 CPU/RAM
- **(U)** - "universal" OpenShift1:4 CPU/RAM
- **(M)** - OpenShift1:8 CPU/RAM

OpenShift VirtualMachineClusterInstanceType OpenShift

VirtualMachineClusterInstanceTypes

VirtualMachineClusterInstanceType OpenShift VM OpenShift

VirtualMachineClusterInstanceTypes spec OpenShift (OpenShift):

- **annotations** - OpenShift

- **cpu** (CPU) - CPU **cpu**
 - **dedicatedCPUPlacement** - VM **cpu** vCPU CPU **cpu**/CPU **cpu** **false**
 - **guest** (CPU) - vCPU **cpu** **0**
 - **isolateEmulatorThread** - VMI **cpu** CPU **cpu**/CPU **cpu** **false**
 - **maxSockets** - **cpu**
 - **model** - VMI **cpu** CPU **cpu** **host-model**
 - **numa** - **cpu** NUMA
 - **guestMappingPassthrough** - **cpu** NUMA **cpu** **cpu** NUMA **cpu** **{}**
 - **realtime** - virt-launcher Pod **cpu** VMI **cpu** (CPU **cpu**)
 - **mask** - vCPU **cpu** libvirt **cpu** (CPU: "0-3,^1","1,2,3","1-2")
- **gpu** - GPU
 - **deviceName** - **cpu** **"**
 - **name** (CPU) - GPU **cpu** **"**
 - **tag** - **cpu**
 - **virtualGPUOptions** - vGPU
 - **display** - vGPU
 - **enabled** - vGPU **cpu** **true**
 - **ramFB** - OS GPU **cpu**
 - **enabled** - **cpu** **true**
- **hostDevices** - **cpu**
 - **deviceName** (CPU) - **cpu** **"**
 - **name** (CPU) - **cpu** **"**
 - **tag** - **cpu**
- **memory** (CPU) - **cpu**
 - **guest** (CPU) - OS **cpu** (CPU: "4096Mi" **cpu** "6Gi")
 - **hugepages** - VMI **cpu** (CPU **cpu**)
 - **pageSize** - **cpu** amd64 **cpu** 1Gi **cpu** 2Mi
 - **maxGuest** - **cpu** **maxGuest** **cpu** **guest**
 - **overcommitPercent** - **cpu** (CPU virt-launcher Pod **cpu** **cpu** **0**)
- **nodeSelector** - **cpu** Kubernetes
 - **schedulerName** - **cpu** Kubernetes **cpu** K8s

VirtualMachineInstanceTypes

`VirtualMachineInstanceType` `VirtualMachineClusterInstanceType` `OpenShift` `VirtualMachine` `specinstancetype` `VirtualMachineInstance` `specinstancetype` `spec.template.spec.domain.cpu` `spec.template.spec.domain.memory` `VirtualMachine` `spec.template.spec.domain.cpu` `spec.template.spec.domain.memory`

`VirtualMachine` `specinstancetype` `VirtualMachineInstance` `specinstancetype` `spec.template.spec.domain.cpu` `spec.template.spec.domain.memory` `VirtualMachine` `specinstancetype` `spec.template.spec.domain.cpu` `spec.template.spec.domain.memory`

`VirtualMachinePreference` `VirtualMachineClusterPreference` `spec` `OpenShift` `VirtualMachine`

`VirtualMachineClusterPreference` `spec` `OpenShift` `VirtualMachineClusterPreference` `spec` `OpenShift` `VirtualMachineClusterPreference` `spec` `OpenShift`

`EFI` `SecureBoot` `VirtualMachineClusterPreference`

```
apiVersion:instancetype.kubevirt.io/v1beta1
kind:VirtualMachineClusterPreference
metadata:
  name: rhel10-custom
spec:
  firmware:
    preferredUseEfi: true
    preferredUseSecureBoot: true
```

`VirtualMachinePreference` `spec` `BIOS` `spec.firmware.preferredUseBios` `SecureBoot` `UEFI`

`CPU`

`VirtualMachinePreference` `1` `OS` `vCPU` `CPU` `vCPU` `CPU` `CPU`

`preferredCPUTopology`

- **sockets** (sockets) - vCPU OS
- **cores** - vCPU OS
- **threads** - vCPU OS
- **spread** - vCPU `spreadOptions`

- **any** - 1 vCPU 1 vCPU (1 vCPU 1 vCPU)

vCPU 1 vCPU

preferredCPUTopology 1 **spread** 1 **spreadOptions** 1 **vCPU**

spreadOptions 2 1:

- **across** - vCPU 1 vCPU
- **ratio** (1:2) - 1 vCPU 1 (1 N 1)

across 1:

- **SocketsCores** (1) - 1 1 N 1 vCPU
- **SocketsCoresThreads** - 1 1 N 2 1 vCPU
- **CoresThreads** (2 1 vCPU 1) - 2 1 vCPU

1 **preferredCPUTopology** 1 **VirtualMachinePreference** 1:

```
apiVersion: instancetype.kubevirt.io/v1beta1
kind: VirtualMachinePreference
metadata:
  name: cpu-topology-example
spec:
  cpu:
    preferredCPUTopology: spread
    spreadOptions:
      across: SocketsCoresThreads
      ratio: 6
```

1 API

1 **VirtualMachine** 1 **VirtualMachineInstance** 1 API 1 VM/VMI 1 Multus CNI 1 NMState Operator

VirtualMachine 1 2 1 (**networks** 1 **interfaces**)

1

VirtualMachine 1 **pod** **spec** (**spec.template.spec**) 1 **networks[]** 1 **kind** 1 **Kubernetes** 1 **VirtualMachine**

1 **spec.template.spec.networks[]** 2 1 1:

- **pod** - pod는 Kubernetes pod (Container Engine)
- **multus** - Multus CNI pod를 위한 CNI plugin (Container Engine)

VirtualMachine

VirtualMachine pod는 VM pod를 위한 CNI plugin (NIC) pod를 위한 CNI plugin Pod pod를 위한 VM pod를 위한 CNI plugin VM pod를 위한 virtio pod를 위한 CNI plugin (OS: Windows) pod를 위한 OpenShift pod를 위한 CNI plugin NIC pod를 위한 CNI plugin

VirtualMachine pod는 spec.domain.devices.interfaces[] pod를 위한 network pod를 위한 CNI plugin

VirtualMachine pod는 spec.domain.devices.interfaces[] pod를 위한 network pod를 위한 CNI plugin (OS: Windows) pod를 위한 OpenShift pod를 위한 CNI plugin NIC pod를 위한 CNI plugin

- **name** (OS) - spec.template.spec.networks[] pod를 위한 CNI plugin
- (OS) - 1 pod를 위한 CNI plugin binding pod를 위한 CNI plugin 1 pod를 위한 CNI plugin binding pod를 위한 CNI plugin:
 - **bridge** - pod를 위한 CNI plugin Linux pod를 위한 CNI plugin
 - **sriov** - vfio pod를 위한 CNI plugin SR-IOV VF pod를 위한 CNI plugin
 - **masquerade** - NAT pod를 위한 CNI plugin IP pod를 위한 CNI plugin (nftables pod를 위한 CNI plugin) pod를 위한 CNI plugin (egress & ingress pod를 위한 CNI plugin)
 - **binding** - pod를 위한 CNI plugin (OS) pod를 위한 CNI plugin:
 - **name** - pod를 위한 CNI plugin 1 pod를 위한 CNI plugin (OS: l2bridge & passt pod를 위한 CNI plugin) pod를 위한 CNI plugin l2bridge pod를 위한 CNI plugin VM interface & network pod를 위한 CNI plugin 2 pod를 위한 CNI plugin passt pod를 위한 CNI plugin OpenShift pod를 위한 CNI plugin
- **model** - pod를 위한 CNI plugin (10/100 pod를 위한 CNI plugin):
 - **e1000** - Intel 82545EM PCI pod를 위한 CNI plugin
 - **e1000e** - Intel 82574 PCI-E pod를 위한 CNI plugin
 - **virtio** - pod를 위한 CNI plugin Linux pod를 위한 CNI plugin
- **macAddress** - pod를 위한 CNI plugin MAC pod를 위한 CNI plugin
- **ports** - pod를 위한 CNI plugin:
 - **name** - pod를 위한 CNI plugin
 - **port** - 0 pod를 위한 CNI plugin 65535 pod를 위한 CNI plugin
- **pciAddress** - pod를 위한 CNI plugin PCI pod를 위한 CNI plugin (OS: 0000:81:00.1)

VirtualMachine pod를 위한 CNI plugin

VirtualMachine pod를 위한 CNI plugin

pod (OS: Windows) OpenShift & Kubernetes pod를 위한 CNI plugin pod를 위한 CNI plugin pod: {} pod를 위한 CNI plugin masquerade: {} pod를 위한 CNI plugin (OS: Windows):

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
```

```

metadata:
  name: default-network-example
  namespace: my-namespace
spec:
  template:
    spec:
      domain:
        devices:
          interfaces:
            - name: default ①
              masquerade: {} ②
      networks:
        - name: default ①
      pod: {} ③

```

① `spec.template.spec` に `domain.devices.interfaces[]` (VM の `spec` 領域) を追加し、`spec` に `networks` を追加し、`name` を指定する。

② `networks` の `interfaces` 配列に `name` と `masquerade` を指定する。IP 変換 (NAT) を有効にする。

③ `pod` 領域を追加する。

関連する概念:

- OpenShift の Kubernetes ネットワーク
- Pod ネットワーク

glossary

OpenShift

CNV

Container Network Virtualization Red Hat OpenShift

NAD

Network Attachment Definition VM

NNCP

Node Network Configuration Policy NMState CR

UDN

User Defined Network 2 3

VLAN

Virtual LAN VLAN

OVS

Open vSwitch

IPAM

IP Address Management IP

PVC

Persistent Volume Claim

DataVolume

CDI CR VM

VMI

Virtual Machine Instance VM

SCC

Security Context Constraint OpenShift

RBAC

Role-Based Access Control Kubernetes

Localnet

Local Network

Cloud-init

VM

Virtctl

VM CLI

□□□□

- [□□](#) - OpenShift
- [□□](#) - □□□□□□□□□□□□□□□□